

Solve Approximate CVP via Variants of Nearest-Colattice

Wenwen Xia¹, Geng Wang^{1*}, and Dawu Gu^{1*}

Shanghai Jiao Tong University
{xww_summer, wanggxx, dwgu}@sjtu.edu.cn

Abstract. The approximate Closest Vector Problem (CVP) is a core computational problem underlying many post-quantum lattice-based signature schemes, including Dilithium, one-more-ISIS, and HuFu. While the security of these schemes is typically expressed in terms of the Inhomogeneous Short Integer Solution (ISIS) problem, it is well-known that ISIS can be efficiently reduced to approximate CVP. Despite its foundational role, approximate CVP with non-negligible approximation factors remains far less explored than other lattice problems such as SVP or LWE, creating a critical gap in both theory and practice.

In this work, we bridge this gap by advancing the Colattice framework for solving approximate CVP with large approximation factors. More concretely, (1) We define a practical version of the Colattice algorithm and propose a randomized Nearest Colattice for generating more than one approximate closest vector. (2) Define a formal strategy space for block-wise approximate CVP. (3) Propose a polynomial-time strategy selection algorithm and prove its correctness under standard lattice heuristics. (4) Building on this, we design an efficient security estimator for approximate CVP in both Euclidean and Infinity norms, and extend it to approximate batch-CVP attack settings. (5) By applying this estimator, we perform concrete security evaluations of Dilithium, HuFu, and one-more-ISIS. Our results reveal that almost none of the evaluated schemes withstand approximate batch-CVP attacks with 2^{32} queries. (6) We integrate a slicer and Colattice into G6K-CPU, leveraging the Locality-Sensitive Hashing (LSH) technique for nearest neighbors search (NNS). This is the first practical implementation of an NNS-accelerated slicer. Our results demonstrate the practical efficiency of approximate CVP and batch-CVP attacks, highlighting the need for more accurate security estimation.

These findings underscore the practical importance of accurate approximate CVP modeling and call for a reassessment of current parameter sets in post-quantum signature schemes.

Keywords: cryptanalysis, approximate CVP, Nearest-Colattice, dimension for free, sieve, slicer

* Corresponding author.

1 Introduction

The approximate Closest Vector Problem (CVP) [1] is a fundamental computational challenge that underpins many forgery attacks on post-quantum lattice-based signature schemes. Notable examples include Dilithium [2], one-more-ISIS [3], HuFu [4], Falcon [5], and Antrag [6]. While the security of these schemes is typically formulated in terms of variants of the Inhomogeneous Short Integer Solution (ISIS) problem, it is important to recognize that ISIS can be reduced to approximate CVP. This reduction highlights the central role of approximate CVP in evaluating the concrete security of these cryptographic constructions.

Formally, given a lattice $\mathcal{L} \subset \mathbb{R}^d$ and a target vector $\mathbf{t} \in \mathbb{R}^d$, the approximate CVP with approximation factor γ (denoted CVP_γ) asks for a lattice vector $\mathbf{v} \in \mathcal{L}$ such that:

$$\|\mathbf{v} - \mathbf{t}\| \leq \gamma \cdot \min_{\mathbf{u} \in \mathcal{L}} \|\mathbf{u} - \mathbf{t}\|.$$

Despite its foundational importance, the study of approximate CVP remains relatively limited compared to other lattice problems such as the Shortest Vector Problem (SVP) or the Learning with Errors (LWE) problem. Most existing research focuses on CVP_γ with $\gamma = O(1)$ [1, 7–9], leaving the complexity and practical implications of larger approximation factors comparatively underexplored. Addressing this gap is essential for both advancing theoretical understanding and providing more accurate security assessments of practical lattice-based schemes.

Prior and related work. A significant breakthrough was achieved with the Nearest-Colattice algorithm (abbreviated as `NColattice`, Alg. 2 in [10]), which introduced the first framework for solving CVP_γ with $\gamma = O(d^c)$. [10] performs a full BKZ- β_0 preprocessing on the lattice basis $\mathbf{B}$, then calls the `NColattice` using an averaged strategy of the form $\llbracket \underbrace{\beta, \dots, \beta}_{\text{repeat } k \text{ times}}, d - k \cdot \beta \rrbracket$ to find the approximate

closest vector to $\mathbf{t}$, where β is a given parameter and $k = \lfloor d/\beta \rfloor$. [10] provided a theoretical foundation for tackling larger approximation factors, but its practical applicability remained uncertain due to the lack of experimental validation and concrete applications in real-world security assessments. The Nearest-Colattice approach demonstrated the feasibility of solving approximate CVP beyond the traditionally studied small factors, yet its implementation and performance in practical scenarios were left as open questions.

Subsequent work in the Restricted SVP paper [11] advanced the field by employing Kannan’s embedding technique to probabilistically reduce approximate CVP to approximate SVP. This reduction enabled the use of well-studied SVP-solving techniques to address approximate CVP instances. Theorem 6.2 in [11] further derived probability formulas for the event that a short vector in the Euclidean norm is also short in the infinity norm, providing a theoretical basis for analyzing the success of such reductions. This methodology was applied to assess the security of the Dilithium signature scheme, bridging the gap between theoretical results and practical cryptographic evaluations.

A summary of the time and space complexities, as well as the applicable cryptographic attacks, for different approximation factors is provided in Table 1.

Table 1. Parameters w.r.t γ . Application versus cryptanalysis of Approx-CVP.

γ	$O(1)$ secure	d^c , c is constant secure	$2^{O(d)}$ insecure
Time Complexity	$2^{0.264d+o(d)}$	$2^{O(\beta)}$	$O(d^2 \cdot 2^{d/2})$
Space Complexity	$2^{0.185d+o(d)}$	$2^{O(\beta)}$	$O(d^2)$
(app)	DSS	DSS	n/a
(attack)	Randomized Slicer	Nearest-Colattice; Restricted SVP Solver	Babai's Nearest Plane

$\beta \in [1, d]$: BKZ blocksize or sieve dimension.

Contribution. Based on the Colattice framework, we define a practical version of the Colattice algorithm and propose a randomized Nearest-Colattice variant capable of generating multiple approximate closest vectors. In Sec. 3, we present the detailed design of both algorithms. We also implement the state-of-the-art randomized slicer for exact CVP within the G6K library using LSH technique and extend it to support both the Colattice algorithm and its randomized variant, enabling practical evaluation of approximate CVP instances with larger approximation factors.¹ Appendix C describes the implementation details and the verification of the correctness for Slicer and Colattice implementation on solving approximate CVP and batch-CVP instances.

Within the Colattice framework, in Sec. 4, we define the approximate CVP block strategy space and prove the existence of an optimal strategy minimizing time cost in this space.

We design a strategy selection algorithm that computes the time-optimal Nearest-Colattice strategy in polynomial time in Sec. 5. Under the Gauss Heuristic and GSA assumptions, we prove the correctness of the selected strategy under the Euclidean norm bound in Sec. 6.

In Sec. 7, we develop a Colattice-based approximate CVP security estimator in two-step mode, and extend this estimator to support both Infinity norm bound and batch-CVP security evaluation with amortized cost considerations.

Using our estimator, we evaluate the security of Dilithium, HuFu, and one-more-ISIS in Sec. 8. Batch-CVP evaluation using Theorem 6 and Table 6 shows

¹ <https://github.com/Summwer/cvp-g6k-cpu-solver>

that none of the evaluated schemes achieve the required gate counts under batch-CVP with 2^{32} queries, highlighting their insufficiency under this attack model.

2 Preliminaries

2.1 Notions

We denote vectors by lower-case bold letters, e.g. $\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots$, and matrices by upper-case bold letters, e.g. $\mathbf{A}, \mathbf{B}, \mathbf{C}, \dots$. For a matrix $\mathbf{B} = (\mathbf{b}_0, \dots, \mathbf{b}_{d-1})$, we write $\mathbf{b}_i$ as its $i + 1$ -th column vector. The Euclidean norm of a vector $\mathbf{v} \in \mathbb{R}^m$ is denoted by $\|\mathbf{v}\|$. We write $\langle \cdot, \cdot \rangle$ for inner products and $\cdot$ for matrix-vector products. By abuse of notation we consider vectors to be row resp. column vectors depending on context, such that $\mathbf{v} \cdot \mathbf{A}$ and $\mathbf{A} \cdot \mathbf{v}$ are both meaningful. We write $\mathbf{0}_{m \times d}$ as the $m \times d$ all zero matrix. If the dimensions are clear from context, we may omit the subscripts. $|\mathbf{B}|$ is denoted as the absolute value of the determinant of matrix $\mathbf{B}$. $A := \llbracket a_1, \dots, a_k \rrbracket$ is denoted as an ordered list with size $\#A = k$ and $A[i] = a_i$, while $\llbracket a_1, \dots, a_{k-1} \rrbracket \cup \llbracket a_k \rrbracket = \llbracket a_1, \dots, a_{k-1}, a_k \rrbracket$. We define $[a, b] := \{a, a + 1, \dots, b - 1\}$ and $[d] = [0, d]$.

Table 2. Symbol Table.

Symbol	Meaning
$\mathcal{R}$	The general reduction as def. 4 in [12]
$\mathcal{C}$	An CVP oracle takes a lattice basis and a target vector $\mathbf{t}$ as input and returns the closest vector in the lattice to $\mathbf{t}$.
Ψ	The blocksize strategy for NColattice (Alg. 3).
$\text{Dist}(\Psi)$	The upper bound of the square distance $\ \mathbf{t} - \mathbf{w}\ ^2$, where $\mathbf{w} \in \mathcal{L}$ output by NColattice (Alg. 3) with blocksize strategy Ψ and $\mathbf{t}$ is the target vector.
$\Psi_1 >_\gamma \Psi_2$ (resp. $<_\gamma$)	Ψ_1 yields larger (resp. smaller) approximate factor than Ψ_2 for same Approx-CVP instance. It implies that $\text{Dist}(\Psi_1) > \text{Dist}(\Psi_2)$ (resp. $\text{Dist}(\Psi_1) < \text{Dist}(\Psi_2)$).

2.2 Lattices

Let $\mathcal{L}$ be a d -dimensional lattice in $\mathbb{R}^m$ and its basis be $\mathbf{B} \in \mathbb{R}^{m \times d}$, we denote $\mathbf{B}_{[i:j]} := (\mathbf{b}_i, \dots, \mathbf{b}_{j-1})$. We can also denote the lattice generated by basis $\mathbf{B}$ as $\mathcal{L}(\mathbf{B})$. Let $\mathbf{B}^* = (\mathbf{b}_0^*, \dots, \mathbf{b}_{d-1}^*)$ be the Gram-Schmidt orthogonalization of $\mathbf{B}$, in which $\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=0}^{i-1} \mu_{i,j} \mathbf{b}_j^*$, $\mu_{i,j} = \langle \mathbf{b}_i, \mathbf{b}_j^* \rangle / \|\mathbf{b}_j^*\|^2$, $\forall i \in [d]$. Denote by l_i the logarithm of Gram-Schmidt norm, i.e. $l_i = \ln(\|\mathbf{b}_i^*\|)$, for $i \in \{0, \dots, d-1\}$. Let $\text{rr}(\mathbf{B}) = (l_0, \dots, l_{d-1})$, abbreviate to rr , $\text{rr}_{[i,j]} = (l_i, \dots, l_{j-1})$. For $i \in [d]$,

we denote by π_i the orthogonal projection over $(\mathbf{b}_0, \dots, \mathbf{b}_{i-1})^\perp$. For $0 \leq j \leq k \leq d-1$, we denote by $\mathbf{B}_{\pi[j:k]}$ the local projected block $(\pi_j(\mathbf{b}_j), \pi_j(\mathbf{b}_{j+1}), \dots, \pi_j(\mathbf{b}_{k-1}))$, and by $\mathcal{L}_{\pi[j:k]}$ the lattice spanned by $\mathbf{B}_{\pi[j:k]}$.

The volume of a lattice $\text{Vol}(\mathcal{L})$ is defined as $\text{Vol}(\mathcal{L}) = |\mathbf{B}| = \prod_{i=0}^{d-1} \|\mathbf{b}_i^*\|$. We write $\lambda_i(\mathcal{L})$ for Minkowski's successive minima, the Gaussian Heuristic predicts $\lambda_1(\mathcal{L}) = \text{GH}(\mathbf{B}) = \sqrt{\frac{d}{2\pi e}} |\mathbf{B}|^{1/d}$. Denote $\text{GH}(d)$ for $\text{GH}(\mathcal{L})$ of any d -dimensional lattice $\mathcal{L}$ of volume 1: $\text{GH}(d) = \sqrt{\frac{d}{2\pi e}}$.

Definition 1 (Voronoi cell of a lattice). *The Voronoi cell of a lattice $\mathcal{L}$ is defined as the region $\mathcal{V} \subset \mathbb{R}^d$ such that $\mathbf{v} \in \mathcal{V}$ iff $\|\mathbf{v}\| \leq \|\mathbf{v} - \mathbf{x}\|$ for all $\mathbf{x} \in \mathcal{L}$. In other words:*

$$\mathcal{V} := \bigcap_{\mathbf{r} \in \mathcal{L}} \mathcal{H}(\mathbf{r}).$$

Definition 2 (Approximate Voronoi cells). *For a lattice $\mathcal{L}$ and a list $L \subseteq \mathcal{L}$, the approximate Voronoi cell generated by L is defined as:*

$$\mathcal{V}_L := \bigcap_{\mathbf{r} \in L} \mathcal{H}(\mathbf{r}).$$

CVP and its Variants [1]. The analysis of lattice algorithms heavily depends on the Gaussian heuristic, as it better represents the practical complexity of the algorithms than their provable counterparts.

As a direct result a random target $\mathbf{t} \in \mathbb{R}^d$ is expected to lie at distance $\lambda_1(\mathcal{L}) \approx \text{GH}(\mathcal{L})$ from the lattice. This gives the following alternative statements for the common variants of the closest vector problem (CVP).

Definition 3 (Closest Vector Problem (CVP)). *Given an arbitrary basis $\mathbf{B}$ for a lattice $\mathcal{L} \subset \mathbb{R}^d$ and a target vector $\mathbf{t} \in \text{span}(\mathcal{L})$. The goal is to find a vector $\mathbf{w} \in \mathcal{L}$ such that $\|\mathbf{w} - \mathbf{t}\| \leq \lambda_1(\mathcal{L})$.*

The hardness of most lattice-based cryptographic schemes actually depends on the following variant.

Definition 4 (Approximate Closest Vector Problem (CVP $_\gamma$)). *For $\gamma = \gamma(d) \geq 1$ (the approximation factor), an arbitrary basis $\mathbf{B}$ for a lattice $\mathcal{L} \subset \mathbb{R}^d$ and a target vector $\mathbf{t} \in \text{span}(\mathcal{L})$. The goal is to find a vector $\mathbf{w} \in \mathcal{L}$ such that $\|\mathbf{w} - \mathbf{t}\| \leq \gamma \cdot \lambda_1(\mathcal{L})$.*

Definition 5 (Batch Approximate Closest Vector Problem (batch-CVP $_\gamma$)). *For $\gamma = \gamma(d) \geq 1$ (the approximation factor), an arbitrary basis $\mathbf{B}$ for a lattice $\mathcal{L} \subset \mathbb{R}^d$ and a set of target vectors $\{\mathbf{t}_1, \dots, \mathbf{t}_Q\} \in \text{span}(\mathcal{L})$. The goal is to find a vector $\mathbf{w}_i \in \mathcal{L}$ such that $\|\mathbf{w}_i - \mathbf{t}_i\| \leq \gamma \cdot \lambda_1(\mathcal{L})$ for each $i \in \{1, \dots, Q\}$.*

2.3 Lattice Reduction

Definition 6. (HKZ reduction and BKZ reduction [13]) The basis $\mathbf{B}$ of a lattice $\mathcal{L}$ is said to be HKZ reduced if $\mathbf{b}_i^* = \lambda_1(\mathcal{L}(\mathbf{B}_{\pi[i:d]}))$, for all $i < d$. It is said BKZ reduced with block-size β (also called as BKZ- β reduced) if $\mathbf{b}_i^* = \lambda_1(\mathcal{L}(\mathbf{B}_{\pi[i:\min\{i+\beta, d\}]})$, for all $i < d$.

Definition 7. (Root Hermite Factor [14]) For a basis $\mathbf{B}$ of d -dimensional lattice, the root Hermite factor is defined as

$$\delta = \left(\|\mathbf{b}_0\| / \text{Vol}(\mathbf{B})^{1/d} \right)^{1/d}, \quad (1)$$

for estimating the quality of the output vector of BKZ. From the test of [14], δ is in range of $(1, 1.021]$, where 1.021 is the root Hermite factor after an LLL reduction. For larger blocksize, it follows the asymptotic formula

$$\delta(\beta)^{2(\beta-1)} = \frac{\beta}{2\pi e} (\beta\pi)^{1/\beta}. \quad (2)$$

Definition 8. (Geometric Series Assumption [15]) Let $\mathbf{B}$ be a BKZ- b reduced basis of a lattice of volume 1. The Geometric Series Assumption states that:

$$\|\mathbf{b}_i^*\| = \alpha_b^{\frac{d-1}{2}-i},$$

where $\alpha_b = \text{GH}(b)^{2/b}$.

2.4 Algorithms for solving CVP and its variants

The Iterative Slicer. The iterative slicer (Algorithm 1) is a simple but effective algorithm that aims to solve CVP or CVP_γ with $\gamma = O(1)$. The preprocessing consists of finding and storing a list $L \subset \mathcal{L}$ of short lattice vectors generated by a lattice sieving algorithm. Then given a target vector $\mathbf{t} \in \mathbb{R}^d$, the iterative slicer repeatedly reduces the target vector $\mathbf{t}$ by subtracting the closest vector $\mathbf{w} \in L$ at each step, until no further reduction is possible (i.e., when $\|\mathbf{t}' - \mathbf{w}\| \geq \|\mathbf{t}'\|$ for all $\mathbf{w} \in L$). The algorithm outputs the lattice vector $\mathbf{t} - \mathbf{t}'$ as an approximate solution to CVP. The quality of the solution depends on the preprocessing list L - when L contains all shortest lattice vectors with size $O(2^{d/2})$, $\mathbf{t}'$ represents the shortest vector in the coset $\mathbf{t} + \mathcal{L}$ and $\mathbf{t} - \mathbf{t}'$ is the exact closest lattice vector to $\mathbf{t}$.

The Randomized Iterative Slicer. However, since generate $O(2^{d/2})$ short lattice vectors becomes impractical in high dimensions, the randomized iterative slicer (Algorithm 2, [8]) circumvents this limitation by employing a smaller preprocessed list $L \subset \mathcal{L}$ combined with randomized restarts: it first samples a random perturbation $\mathbf{t}'$ from the coset $\mathbf{t} + \mathcal{L}$, then applies the iterative slicer (Algorithm 1) to reduce $\mathbf{t}'$ to a lattice vector $\mathbf{w} \in \mathcal{L}$, and finally checks whether

input : A target vector $\mathbf{t} \in \mathbb{R}^d$, a short vector list $L \subset \mathcal{L}$;
output: The close vector $\mathbf{w} \in \mathcal{L}$;
1 Function IterativeSlicer($\mathbf{t}, L$):
2 $\mathbf{t}' \leftarrow \mathbf{t}$;
3 **repeat**
4 | $\mathbf{t}' \leftarrow \mathbf{t}' - \mathbf{w}$, for $\mathbf{w}$ satisfies $\min_{\mathbf{w} \in L} \|\mathbf{t}' - \mathbf{w}\|$;
5 **until** $\|\mathbf{t}' - \mathbf{w}\| \geq \|\mathbf{t}'\|$, $\forall \mathbf{w} \in L$;
6 **return** $\mathbf{t} - \mathbf{t}'$;

Algorithm 1: Iterative Slicer [7]

$\|\mathbf{t} - (\mathbf{w} + (\mathbf{t} - \mathbf{t}'))\| \leq \gamma \cdot \text{GH}(\mathcal{L})$; this process repeats until success. While each iteration may succeed with only modest probability, multiple independent trials guarantee high overall success probability without requiring an exponentially large list L , as quantitatively analyzed by Ducas et al. [1] who characterized the precise trade-offs between the list size, approximation factor γ , and required number of repetitions.

input : A target vector $\mathbf{t} \in \mathbb{R}^d$, a short vector list $L \subset \mathcal{L}$, a target distance ratio $\gamma \in \mathbb{R}$;
output: The close vector $\mathbf{w} \in \mathcal{L}$ s.t. $\|\mathbf{t} - \mathbf{w}\| \leq \gamma \text{GH}(\mathcal{L})$;
1 Function RandSlicer($\mathbf{t}, L, \gamma$):
2 **repeat**
3 | $\mathbf{t}' \leftarrow \text{Sample}(\mathbf{t} + \mathcal{L})$;
4 | $\mathbf{w} \leftarrow \text{IterativeSlicer}(\mathbf{t}', L)$;
5 **until** $\|\mathbf{t} - \mathbf{w}\| \leq \gamma \text{GH}(\mathcal{L})$;
6 **return** $\mathbf{w} + (\mathbf{t} - \mathbf{t}')$;

Algorithm 2: Randomized Iterative Slicer [8]

3 CVP $_{\gamma}$ Solver based on Nearest-Colattice

In this section, we first present the Nearest-Colattice algorithm and describe its practical version in Section 3.1. Building upon this foundation, we then propose an enhanced version of the algorithm to accelerate the efficiency of solving CVP $_{\gamma}$ in Section 3.3.

3.1 Nearest-Colattice Algorithm

The Nearest-Colattice Algorithm (abbreviated as NColattice, Alg. 2 in [10]) was proposed in [10]. It gave a new algorithm to solve CVP $_{\gamma}$. Assume there is a d dimensional lattice $\mathcal{L}$ and a target vector $\mathbf{t}$. The NColattice firstly constructs a filtration $\{0\} = \mathcal{L}_0 \subset \mathcal{L}_1 \subset \mathcal{L}_2 \subset \cdots \subset \mathcal{L}_k = \mathcal{L}$ of lattice $\mathcal{L}$ with $0 = \text{rk}(\mathcal{L}_0) < \text{rk}(\mathcal{L}_1) < \text{rk}(\mathcal{L}_2) < \cdots < \text{rk}(\mathcal{L}_k) = \text{rk}(\mathcal{L})$, where $\mathcal{L}_{i-1}$ is the sublattice of $\mathcal{L}_i$ for

i from 1 to k . Then, it iteratively calls a CVP solver to find the close vector to the projected vector of the target vector $\mathbf{t}$ on a quotient lattice $\mathcal{L}_i/\mathcal{L}_{i-1}$, from $i = k$ to 1.

We present a more practical version as Alg. 3 shown, which offers significant concretization over the original schematic description. Unlike the abstract formulation, our version explicitly specifies all input parameters including the lattice basis $\mathbf{B}$, stratification strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$ with $\sum_{i=1}^k \beta_i = d$, and a CVP oracle $\mathcal{C}$. The projection mechanics are rigorously defined through the operation $\mathbf{t}_i = \mathbf{t} - \pi_r(\mathbf{t}) + \pi_r(\mathbf{w}_{i+1}) - \mathbf{w}_{i+1}$, replacing the original implicit quotient space reduction, while the basis reconciliation process now explicitly transforms quotient-space solutions $\mathbf{w}_i^{(\pi)}$ back to the original lattice via basis operations $\mathbf{B}_{[l:r]}\mathbf{x}$. If $\beta_i = 1$, then we can regard it as a dimension-for-free operation and call the babai's Nearest Plane Algorithm to find the closest vector on the 1-dimensional projected sub-lattice. If $\beta_i > 1$, we will firstly generate the short vector set with size $|L| = \alpha^{\beta_i}$ and then call the Randomized iterative Slicer Algorithm (i.e. the function **RandSlicer**) to find the closest vector on the projected sub-lattice.

The algorithm dynamically manages coordinate ranges through tracked variables l and r for transparent filtration handling. These enhancements collectively provide both theoretical clarity and practical implementability.

input : A target vector $\mathbf{t} \in \mathbb{R}^d$, a lattice basis $\mathbf{B}$, a strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$
with $\sum_{i=1}^k \beta_i = d$, a CVP oracle $\mathcal{C}$
output: The vector $\mathbf{w} \in \mathcal{L}$ close to $\mathbf{t}$;

```

1 Function NColattice( $\mathbf{t}, \mathbf{B}, \Psi, \mathcal{C}$ ):
2    $\mathbf{t}' \leftarrow \mathbf{t}; l \leftarrow d; r \leftarrow d;$ 
3   for  $i$  from  $k$  to 1 do
4      $l \leftarrow l - \beta_i;$ 
5      $\mathbf{t}_i \leftarrow \mathbf{t} - \pi_r(\mathbf{t}) + \pi_r(\mathbf{w}_{i+1}) - \mathbf{w}_{i+1};$ 
6      $\mathbf{w}_i^{(\pi)} \leftarrow$  run a CVP call  $\mathcal{C}$  on  $\mathbf{B}_{\pi[l:r]}$  and  $\pi[l:r](\mathbf{t}_i);$ 
7      $\mathbf{x}^T \leftarrow \mathbf{w}_i^{(\pi)}$  w.r.t  $\mathbf{B}_{\pi[l:r]}$ ;  $\mathbf{w}_i \leftarrow \mathbf{B}_{[l:r]}\mathbf{x};$ 
8      $\mathbf{t}' \leftarrow \mathbf{t}_i - \mathbf{w}_i;$ 
9      $r \leftarrow l;$ 
10  return  $\mathbf{t} - \mathbf{t}';$ 

```

Algorithm 3: Nearest-Colattice [10]

3.2 Randomized Nearest-Colattice

In some cases, we need to output multiple approximate closest vectors for a given target vector. To address this requirement, we propose the Randomized Nearest-Colattice algorithm (**RandNColattice**, Algorithm 4). While the original nearest-colattice approach relies on exact CVP oracle calls for all projected sublattices, our **RandNColattice** algorithm employs γ_1 -approximate solutions throughout all sublattices, significantly reducing the computational cost of each CVP call.

input : A target vector $\mathbf{t} \in \mathbb{R}^d$, a lattice basis $\mathbf{B}$, a strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$ with $\sum_{i=1}^k \beta_i = d$, a CVP_{γ_1} oracle $\mathcal{C}$, an approximate factor γ_1 ; Size of the approximate close vector set l_W ;

output: An approximate close vector set $W \subset \mathcal{L}$;

```

1 Function RandNColattice( $\mathbf{t}, \mathbf{B}, \Psi, \mathcal{C}, \gamma_1, l_W$ ):
2    $\mathbf{t}' \leftarrow \mathbf{t}; l \leftarrow d; r \leftarrow d;$ 
3   for  $i$  from  $k$  to 2 do
4      $l \leftarrow l - \beta_i;$ 
5      $\mathbf{t}_i \leftarrow \mathbf{t} - \pi_r(\mathbf{t}) + \pi_r(\mathbf{w}_{i+1}) - \mathbf{w}_{i+1};$ 
6      $\mathbf{w}_i^{(\pi)} \leftarrow$  run a  $\text{CVP}_{\gamma_1}$  call  $\mathcal{C}$  on  $\mathbf{B}_{\pi[l:r]}$  and  $\pi_{[l:r]}(\mathbf{t}_i)$ ;
7      $\mathbf{x}^T \leftarrow \mathbf{w}_i^{(\pi)}$  w.r.t  $\mathbf{B}_{\pi[l:r]}$ ;  $\mathbf{w}_i \leftarrow \mathbf{B}_{[l:r]}\mathbf{x};$ 
8      $\mathbf{t}' \leftarrow \mathbf{t}_i - \mathbf{w}_i;$ 
9      $r \leftarrow l;$ 
10   $\mathbf{t}_1 \leftarrow \mathbf{t} - \pi_{\beta_1}(\mathbf{t}) + \pi_{\beta_1}(\mathbf{w}_2) - \mathbf{w}_2; W_1 \leftarrow \emptyset; W \leftarrow \emptyset;$ 
11  for  $\#W_1 < l_W$  or no more vector is within approximate factor  $\gamma_1$  do
12     $\mathbf{t}'_1 \leftarrow$  sample from  $\pi_{[0:\beta_1]}(\mathbf{t}_1) + \mathcal{L}(\mathbf{B}_{\pi[0:\beta_1]});$ 
13     $\mathbf{w}_1^{(\pi)} \leftarrow$  run an approximate  $\text{CVP}_{\gamma_1}$  call  $\mathcal{C}$  on  $\mathbf{B}_{\pi[0:\beta_1]}$  and  $\mathbf{t}'_1;$ 
14     $W_1 \leftarrow W_1 \cup \{\mathbf{w}_1^{(\pi)}\};$ 
15  for  $\mathbf{w}_1^{(\pi)} \in W_1$  do
16     $\mathbf{x}^T \leftarrow \mathbf{w}_1^{(\pi)}$  w.r.t  $\mathbf{B}_{\pi[0:\beta_1]}$ ;  $\mathbf{w}_i \leftarrow \mathbf{B}_{[0:\beta_1]}\mathbf{x}; \mathbf{t}' \leftarrow \mathbf{t}_1 - \mathbf{w}_i;$ 
17     $W \leftarrow W \cup \{\mathbf{t} - \mathbf{t}'\};$ 
18  return  $W;$ 

```

Algorithm 4: Randomized Nearest-Colattice

The use of a CVP_{γ_1} oracle naturally yields multiple candidate vectors in each sub-block computation due to the property of the approximate Voronoi cell (Sec. 7.3), enabling the generation of diverse solutions.

The algorithm takes as input a target vector $\mathbf{t} \in \mathbb{R}^d$, a lattice basis $\mathbf{B}$, a strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$ where $\sum_{i=1}^k \beta_i = d$, a CVP_{γ_1} oracle $\mathcal{C}$, and a parameter l_W specifying the desired number of output vectors. It operates through three main phases: first processing blocks from β_k to β_2 by successively projecting the target and computing approximate closest vectors, then sampling multiple candidate solutions for the first block β_1 to ensure solution diversity, and finally reconstructing the complete approximate closest vectors by combining the partial solutions from all blocks. The algorithm returns a set $W \subset \mathcal{L}$ containing l_W distinct lattice vectors that are γ_1 -approximate solutions to the CVP instance, with the randomization in the first block ensuring both solution quality and diversity.

3.3 Nearest-Colattice in Two-step Mode

Given a generic lattice reduction algorithm $\mathcal{R}$ defined as Def. 4 in [12] and a CVP oracle $\mathcal{C}$, we design a more general approach for CVP_{γ} through the **TwoStepNColattice** algorithm (Algorithm 5) by inputting a two-step strategy $S \in \mathcal{S}(\mathcal{R}, \mathcal{C})$ (Def. 9), which considers the **NColattice** method with a progressive

lattice reduction preprocessing. This general approach encompasses the special case mentioned above.

To further improve performance, we design a strategy search algorithm to find an optimized strategy of **TwoStepNColattice**. This strategy is then used to find a lattice vector that is close to the target vector $\mathbf{t}$. In Sec. 4.1, we define the strategy space for this generalized scenario. Then, in Sec. 4.2, we present an intuitive strategy search algorithm aimed at minimizing the time-cost for the **TwoStepNColattice**.

4 Two-step Nearest-Colattice Strategy Space and the Strategy Search Algorithm

4.1 Two-step Nearest-Colattice strategy Space

We defined the Two-step Nearest-Colattice strategy using the same approach as the definition of the strategy space in [12]. The Two-step Nearest-Colattice strategy space using **NColattice** Algorithm for the Approximate CVP searching step can be defined as follows,

Definition 9 (Two-step Nearest-Colattice strategy Space). *Given a d -dimensional lattice basis $\mathbf{B}$ for a lattice $\mathcal{L}$, a target vector $\mathbf{t}$ and the approximate factor γ of a CVP_γ instance. Suppose we have a lattice reduction algorithm $\mathcal{R}$ with an adjustable parameter ξ , a CVP Oracle $\mathcal{C}$, define $\mathcal{S}(\mathcal{R}, \mathcal{C})$ as the **Two-step Nearest-Colattice strategy Space** for the combination of the lattice reduction algorithm and oracles. Assume that*

$$\sum_{i=1}^k \beta_i = d, \quad (3)$$

where $\beta_i, k \in \mathbb{Z}^*$. For each Two-step Nearest-Colattice strategy

$$\mathbf{S} = (\Xi = [\xi_0, \dots, \xi_i, \dots], \Psi = [\beta_1, \dots, \beta_k]) \in \mathcal{S}, \quad (4)$$

it can find the close vector $\mathbf{w} \in \mathcal{L}$ such that the distance $\Delta = \|\mathbf{t} - \mathbf{w}\|$ satisfies $\Delta \leq \gamma \cdot \lambda_1(\mathcal{L})$ by running the following two steps:

1. *Reduction Step:* Iteratively call the reduction method $\mathcal{R}$ with parameter ξ_i on lattice basis $\mathbf{B}$ using the reduction strategy Ξ , where each $\mathcal{R}$ - ξ can improve the basis quality. Output a lattice basis $\mathbf{B}'$ after running all the reductions with parameter $\xi \in \mathbf{S}$.
2. *Approximate CVP Searching Step:* Find the approximate close vector $\mathbf{w} \in \mathcal{L}$ such that the distance $\Delta = \|\mathbf{t} - \mathbf{w}\| \leq \gamma \cdot \lambda_1(\mathcal{L})$ by running **NColattice**($\mathbf{t}, \mathbf{B}', \Psi, \mathcal{C}$).

According to Theorem 1 in [12], the selection range for Ξ is finite. Building on this result, along with the finite selection range of Ψ , we can prove that the Two-step Nearest-Colattice strategy Space is finite. Therefore, we establish Theorem 1 as the theoretical foundation for the subsequent strategy search algorithms.

Theorem 1. *Two-step Nearest-Colattice strategy Space $\mathcal{S}$ as Def. 9 is finite.*

input : A target vector $\mathbf{t} \in \mathbb{R}^d$, lattice basis $\mathbf{B}$, Strategy $\mathbf{S}$; a lattice reduction algorithm $\mathcal{R}$, a CVP call $\mathcal{C}$, a two-step strategy $\mathbf{S} = (\Xi, \Psi) \in \mathcal{S}(\mathcal{R}, \mathcal{C})$;
output: $\mathbf{w} \in \mathcal{L}(\mathbf{B})$ s.t. $\|\mathbf{t} - \mathbf{w}\| \leq \gamma \cdot \text{GH}(\mathcal{L})$;
1 Function TwoStepNColattice($\mathbf{t}, \mathbf{B}, \mathbf{S}$):
2 $(\Xi, \Psi) \leftarrow \mathbf{S}$;
3 $\mathbf{B} \leftarrow \text{LLL}(\mathbf{B})$;
4 **for** $\xi \in \Xi$ **do**
5 $\mathbf{B} \leftarrow \mathcal{R}(\mathbf{B}, \xi)$;
6 $\mathbf{w} \leftarrow \text{run NColattice}(\mathbf{t}, \mathbf{B}, \Psi, \mathcal{C})$;
7 **return** $\mathbf{w}$;

Algorithm 5: Nearest-Colattice in Two-step Mode.

Proof. As the definition of the Two-step Nearest-Colattice strategy Space, we have $\sum_{i=1}^k \beta_i = d$. For each blocksize number k , we can depart the d dimension into k parts by $k - 1$ separators. So it has C_{d-1}^{k-1} choices. Sum up all cases for different values of k , we get that $\sum_{k=1}^d C_{d-1}^{k-1} = 2^{d-1}$. Since the possible choices of the reduction preprocess is finite, it infers that the Two-step Nearest-Colattice strategy Space is finite. $\square$

4.2 Two-step Nearest-Colattice strategy Search Algorithm

Given a time cost model $\mathcal{T} = (T_{\mathcal{R}}, T_{\mathcal{C}})$ and a strategy $\mathbf{S} = (\Xi, \Psi)$, denote the time cost of entire reduction step as $T_{\mathcal{R}s}(\Xi) = \sum_{\xi \in \Xi} T_{\mathcal{R}}(\xi)$ and the time cost of Approximate CVP search step as $T_{\text{NC}}(\Psi) = \sum_{i=1}^k T_{\mathcal{C}}(\beta_i)$, where $T_{\mathcal{C}}(\cdot)$ is the time cost of a CVP call. Then, the total time cost of the strategy $\mathbf{S} = (\Xi, \Psi)$ for solving a CVP $_{\gamma}$ instance is $T_{\text{total}}(\mathbf{S}) = T_{\mathcal{R}s}(\Xi) + T_{\text{NC}}(\Psi)$.

According to Theorem 1, the Two-step Nearest-Colattice strategy Space $\mathcal{S}$ for solving a CVP $_{\gamma}$ instance is finite. There exists a strategy $\mathbf{S}_{\min}$ s.t. $T_{\text{total}}(\mathbf{S}_{\min}) = \min\{T_{\text{total}}(\mathbf{S}) : (\mathbf{S}) \in \mathcal{S}\}$. To find $\mathbf{S}_{\min}$, we can formalize the combinatorial optimization problem as the following:

Definition 10 (Problem to Search a Refined Colattice Strategy). *Given a CVP $_{\gamma}$ instance $(\mathbf{B}, \mathbf{t}, \gamma)$, consider an arbitrary lattice reduction algorithm $\mathcal{R}$, a CVP oracle $\mathcal{C}$ and their corresponding time cost model $\mathcal{T}$. The objective is finding a refined strategy $\mathbf{S}_{\min} \in \mathcal{S}$ for TwoStepNColattice to minimize the total time cost for solving a given CVP $_{\gamma}$ instance under a given time cost model $\mathcal{T}$.*

In this paper, we propose an algorithm named *Nearest-Colattice Strategy Search* (NCSSearch) to identify a strategy $\mathbf{S}_{\min} \in \mathcal{S}$ that minimizes the time cost for solving an arbitrary LWE instance under the given reduction algorithm $\mathcal{R}$ and the CVP oracle $\mathcal{C}$.

To accurately estimate the reduction effort for each strategy, we introduce two key components. The first is a polynomial-time simulator for $\mathcal{R}$, referred to as **ReductionSim**. The second is an output norm prediction for **NColattice**, bounded by the inequality (5) from Theorem 2.

`ReductionSim` can be initialized as one of the simulators: the BKZ simulator [14] or the PnJBKZ simulator proposed in [12], depending on the reduction algorithm $\mathcal{R}$ used, such as BKZ or PnJBKZ. Given the initial Gram-Schmidt norms rr_0 and a reduction strategy Ξ , `ReductionSim`(rr_0, Ξ) predicts how the Gram-Schmidt norms changes after iteratively calling $\mathcal{R}$ - ξ reduction for $\xi \in \Xi$ with input rr_0 . If Ξ contains a single reduction strategy ξ , we can also re-write `ReductionSim`(rr_0, Ξ) as `ReductionSim`(rr_0, ξ).

Condition 1 (The Condition for NCSSearch.) *Given a lattice reduction algorithm $\mathcal{R}$ and a CVP oracle $\mathcal{C}$, we have*

1. A reduction simulator named `ReductionSim` can predict the reduction effort of $\mathcal{R}$ accurately in polynomial time cost;
2. A norm estimation method that can estimate the output norm of `NColattice` for a given strategy Ψ ;
3. The time cost model $\mathcal{T}$ for $\mathcal{R}$ and $\mathcal{C}$;

If Condition 1 is satisfied, we give a simplified NCSSearch algorithm (abbreviated as `SimNCSSearch`, Alg. 6) that can be utilized to determine the minimum time-cost strategy $S_{\min}$ for `TwoStepNColattice`. This is achieved by systematically enumerating all possible strategies and identifying the one with the lowest time cost.

input : A lattice basis $\mathbf{B}$, a target distance ratio $\gamma \in \mathbb{R}$, a simulator `ReductionSim` for $\mathcal{R}$, a time cost model $\mathcal{T}$;
output: The minimum time-cost strategy $S_{\min}$;

```

1 Function SimNCSSearch( $\mathbf{t}, \mathbf{B}, \gamma$ ):
2    $(T_{\mathcal{R}}, T_{\mathcal{C}}) \leftarrow \mathcal{T}$ ;  $T_{\min} \leftarrow T_{\mathcal{C}}(d)$ ;
3    $rr_0 \leftarrow$  The Gram-Schmidt norms of the lattice basis  $\mathbf{B}$ ;
4    $S_{\min} \leftarrow ([\mathbf{I}], [d])$ ;
5   for each possible strategy  $(\Xi, \Psi) \in \mathcal{S}$  do
6      $rr \leftarrow$  ReductionSim( $rr_0, \Xi$ );
7      $\Delta \leftarrow$  The predicted output norm of NColattice by  $rr$  and  $\Psi$ ;
8      $T_{\text{total}} \leftarrow T_{\mathcal{R}}(\Xi) + T_{\text{NC}}(\Psi)$ ;
9     if  $\Delta \leq \gamma \cdot \lambda_1(\mathcal{L})$  and  $T_{\text{total}} < T_{\min}$  then
10       $T_{\min} \leftarrow T$ ;  $S_{\min} \leftarrow S$ ;
11  return  $S_{\min}$ ;
```

Algorithm 6: Simplified NCSSearch

Since the strategy space $\mathcal{S}$ is finite according to Theorem 1 and the time overhead is uniquely determined by `TwoStepNColattice`'s strategy S . Then, Corollary 1 holds apparently.

Corollary 1 *If Condition 1 is satisfied, `SimNCSSearch` outputs a minimum time-cost strategy of `TwoStepNColattice` for solving a given CVP $_{\gamma}$ instance.*

5 A Polynomial Nearest-Colattice Strategy Search Algorithm

Although the `SimNCSSearch` algorithm can output the minimum time-cost strategy for `TwoStepNColattice`, the vast strategy space makes it challenging to quickly identify the optimal solution. To address this issue, we propose a strategy search algorithm with polynomial time cost that can efficiently find the minimum time-cost strategy for `TwoStepNColattice`, and we prove its correctness using the Gaussian Heuristic (GH) and the GSA.

While the pruning strategy for reduction strategy Ξ is discussed in [12], this section focuses on selecting the `NColattice` strategy X_2 , which is achieved through the `NCSSearch` algorithm (Alg. 7). The key idea behind the `NCSSearch` algorithm is to iteratively refine the strategy by narrowing down the search space based on a series of heuristic checks. The algorithm begins with a greedy initialization that selects a reasonable starting point, then it applies a series of pruning rules to discard strategies that are unlikely to achieve the desired approximation factor. In Sec. 6, we provide a formal proof of the algorithm’s correctness, based on the Gaussian Heuristic and GSA.

The `NCSSearch` algorithm operates in polynomial time with respect to the lattice dimension, ensuring that it can efficiently handle high-dimensional lattice instances. By employing a combination of greedy initialization and adaptive refinement, `NCSSearch` significantly reduces the search space, providing a scalable approach to finding cost-optimal strategies for `NColattice`. In the following, we describe the details of the `NCSSearch` algorithm and demonstrate its effectiveness in finding the minimum time-cost strategy.

5.1 Main Algorithm

In this section, we present a detailed description of our proposed `NCSSearch` algorithm (Alg. 7), which efficiently searches for a blocksize strategy that minimizes the time cost of the `NColattice` procedure. Given a reduced lattice basis with Gram–Schmidt norms rr , and a prescribed approximation factor $\gamma \in \mathbb{R}$, the algorithm aims to determine a sequence of block sizes $\Psi_{\min}$ that enables `NColattice` to find an approximate closest vector $\mathbf{w} \in \mathcal{L}$ to a target $\mathbf{t} \in \mathbb{R}^d$, such that $\|\mathbf{t} - \mathbf{w}\| \leq \gamma \cdot \text{GH}(\mathcal{L})$, while minimizing the total cost $T_{\text{NC}}(\Psi)$.

The algorithm begins with a greedy search over evenly-partitioned strategies $\Psi = \text{AvgStrategy}(d, k_0)$ for k_0 from 2 to d . The function `AvgStrategy`(d, k) constructs a strategy of k blocks as

$$\text{AvgStrategy}(d, k) = \underbrace{[\lceil d/k \rceil, \dots, \lceil d/k \rceil]}_{d \bmod k}, \underbrace{[\lfloor d/k \rfloor, \dots, \lfloor d/k \rfloor]}_{k - (d \bmod k)},$$

ensuring a nearly balanced partition of the dimension d . For each such candidate, the algorithm estimates the projection length Δ using Eq. (5) and selects the first Ψ satisfying $\Delta \leq \gamma \cdot \text{GH}(rr)$. If no such strategy is found during this initialization, the search interval for refinement is established using the last valid and first invalid strategies.

input : The Gram-Schmidt norms rr of the lattice basis $\mathbf{B}$, a target distance ratio $\gamma \in \mathbb{R}$;

output: The minimum time-cost strategy $\Psi_{\min}$;

```

1 Function NCSSearch( $rr, \gamma$ ):
2    $\Psi_{\min} \leftarrow$  the minimal time cost solvable strategy of the original NColattice;
3    $T_{\min} \leftarrow T_{\text{NC}}(\Psi_{\min})$ ;
4   for  $k_0$  from 2 to  $d$  do
5      $\Psi \leftarrow \text{AvgStrategy}(d, k_0)$ ;
6      $\Delta \leftarrow$  Eq. (5) with inputs  $rr$  and strategy  $\Psi$ ;
7     if  $\Delta \leq \gamma \cdot \text{GH}(rr)$  then
8        $\Psi_{\min} \leftarrow \Psi$ ;  $T_{\min} \leftarrow T_{\text{NC}}(\Psi)$ ;
9     else
10       $\beta_{\max} \leftarrow \Psi_{\min}[0]$ ;  $\beta_{\min} \leftarrow \Psi[0]$ ;
11      break;
12  for  $k$  from  $k_0$  to  $d$  do
13     $\Psi_{\min}, T_{\min} \leftarrow \text{SSearchLoop}(\Psi_{\min}, T_{\min}, \Psi_F, k, \beta_{\max}, \beta_{\min}, \gamma, rr, b_{\min})$ ;
14  return  $\Psi_{\min}$ ;

```

Algorithm 7: Nearest-Colattice Strategy Search

To refine the strategy further, the algorithm invokes the recursive subroutine Strategy Search Loop (Abbreviate as **SSearchLoop**, Alg. 8) for each $k \geq k_0$. This function performs a bisection search over feasible blocksize ranges to construct a minimal-cost strategy while preserving the approximation guarantee. At each recursive step, two candidate strategies, $\Psi_{\min}$ and $\Psi_{\max}$, are formed as lower and upper bounds. If neither satisfies the constraint, the algorithm recurses on the next block until a full-length, valid strategy is built.

The time complexity of the **SSearchLoop** is $O(k \cdot \log d)$, as the blocksize range $\beta_{\max} - \beta_{\min}$ can be conservatively bounded by the lattice dimension d . Since this loop is executed for all $k \in [k_0, d]$, the total time complexity of the overall **NCSSearch** procedure becomes

$$O\left(\sum_{k=k_0}^d k \cdot \log d\right) = O(d^2 \cdot \log d).$$

This confirms that **NCSSearch** runs in polynomial time with respect to the lattice dimension, ensuring scalability for high-dimensional lattice instances. By combining a greedy initialization with an adaptive recursive refinement routine, the algorithm efficiently explores the space of blocksize strategies to identify the one minimizing the total NColattice cost under a given approximation factor γ .

6 Correctness Proof for NCSSearch

Given a CVP_γ instance defined as Def. 3, all derivations in this section are based on some or all of the following three assumptions: (1) the Gaussian Heuristic, (2)

input : Current minimum time-cost strategy $\Psi_{\min}$, minimum time cost $T_{\min}$, the fixed blocks Ψ_F , strategy size k , the maximum blocksize for current index $\beta_{\max}$, minimum blocksize for current index $\beta_{\min}$, the target distance ratio γ , GS-lengths rr , the lower bound of each block $b_{\min}$;

output: Current minimum time-cost strategy $\Psi_{\min}$, minimum time cost $T_{\min}$;

```

1 Function SSearchLoop( $\Psi_{\min}, T_{\min}, \Psi_F, k, \beta_{\max}, \beta_{\min}, \gamma, rr, b_{\min}$ ):
2   //Use Bisection method in each loop.
3   while  $\beta_{\max} \geq \beta_{\min}$  do
4      $\beta \leftarrow \lfloor \frac{\beta_{\min} + \beta_{\max}}{2} \rfloor$ ;  $d \leftarrow \text{len}(rr) - \sum_{\beta \in \Psi_F} \beta$ ;  $\text{it} \leftarrow \#\Psi_F + 1$ ;  $k_{\text{rem}} \leftarrow k - \text{it}$ ;
       $\text{left} \leftarrow \lfloor \frac{d - k_{\text{rem}} \cdot b_{\min}}{\beta - b_{\min}} \rfloor$ ;  $\text{mid} \leftarrow d - \text{left} \cdot \beta - b_{\min} \cdot (k_{\text{rem}} - \text{left} - 1)$ ;
5      $\Psi_{\text{cmin}} \leftarrow \Psi_F \cup \llbracket \beta \rrbracket \cup \text{AvgStrategy}(d - \beta, k_{\text{rem}} - 1)$ ;
6      $\Psi_{\text{cmax}} \leftarrow \Psi_F \cup \llbracket \underbrace{\beta, \dots, \beta}_{\text{repeat left times}}, \text{mid}, \underbrace{b_{\min}, \dots, b_{\min}}_{\text{repeat } k_{\text{rem}} - \text{left} - 1 \text{ times}} \rrbracket$ ;
7      $\Delta_{\text{cmin}}$  (resp.  $\Delta_{\text{cmax}}$ )  $\leftarrow$  Eq. (5) with  $rr$  and  $\Psi_{\text{cmin}}$  (resp.  $\Psi_{\text{cmax}}$ );
8     if  $\Delta_{\text{cmax}} > \gamma \cdot \text{GH}(rr)$  then
9        $\beta_{\min} \leftarrow \beta + 1$ ;
10    else if  $\Delta_{\text{cmin}} \leq \gamma \cdot \text{GH}(rr)$  then
11       $\beta_{\max} \leftarrow \beta$ ;
12      if  $T_{\text{NC}}(\Psi_{\text{cmin}}) < T_{\min}$  then
13         $\Psi_{\min} \leftarrow \Psi_{\text{cmin}}$ ;  $T_{\min} \leftarrow T_{\text{NC}}(\Psi)$ ;
14      if  $\beta_{\max} = \beta_{\min}$  then
15        return  $\Psi_{\min}, T_{\min}$ ;
16    else
17       $\Psi_{\min}, T_{\min} \leftarrow \text{SSearchLoop}(\Psi_{\min}, T_{\min}, \Psi_F \cup \llbracket \beta \rrbracket, k, \Psi_{\text{cmax}}[\text{it} + 1], \Psi_{\text{cmin}}[\text{it} + 1], \gamma, rr, b_{\min})$ ;
18       $\beta_{\max} \leftarrow \beta$ ;
19      if  $\beta_{\max} = \beta_{\min}$  then
20        return  $\Psi_{\min}, T_{\min}$ ;
21  return  $\Psi_{\min}, T_{\min}$ ;

```

Algorithm 8: Strategy Search Loop

the GSA, under which the root Hermite factor of the lattice is denoted by δ and the Gram-Schmidt norms generated by GSA is $(\delta^d, \delta^{d \cdot \frac{d-3}{d-1}}, \dots, \delta^{d \cdot \frac{d-1-2i}{d-1}}, \dots, \delta^{-d})$ with $\text{Vol}(\mathcal{L}) = 1$, and (3) the existence of a CVP oracle $\mathcal{C}$ returns a lattice vector that is within a distance of $\text{GH}(\mathcal{L})$ from the target vector. To avoid redundancy, these assumptions will be omitted in the statements of theorems and lemmas in this section.

6.1 Distance Bound and Time Cost Model

In order to systematically analyze and compare the effectiveness of various `NColattice` strategies, we adhere to Condition 1, which requires knowledge of both the output norm distance and the associated computational cost. Accordingly, Theorem 2 is dedicated to formulating a precise method for estimating the norm distance, as well as proposing a runtime cost model for each strategy.

Theorem 2. *Given an `NColattice` strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$, the distance between the target vector $\mathbf{t}$ and the output $\mathbf{w}$ produced by `NColattice` satisfies the following bound:*

$$\|\mathbf{t} - \mathbf{w}\|^2 \leq \text{Dist}(\Psi) := \sum_{i=1}^k \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}, d_i]}), \quad (5)$$

where $d_i = \sum_{j=1}^i \beta_j$ and $d_0 = 0$.

Furthermore, if the time cost of CVP oracle in d dimension is $2^{c \cdot d}$ for some constant $0 < c < 1$, then the time cost of executing a `NColattice` tour with strategy Ψ is

$$T_{\text{NC}}(\Psi) = \sum_{i=1}^k 2^{c \cdot \beta_i}. \quad (6)$$

Proof. From the process of Alg. 3,

$$\begin{aligned} \|\mathbf{t} - \mathbf{w}\|^2 &\leq \left\| \mathbf{t} - \sum_{i=1}^k \mathbf{w}_i \right\|^2 \\ &= \left\| \pi_{[l_k:r_k]}(\mathbf{t}) - \mathbf{w}_k^{(\pi)} + \pi_{[0:l_k]}(\mathbf{t}) + \pi_{[0:l_k]}(\mathbf{w}_k) - \sum_{i=1}^{k-1} \mathbf{w}_i \right\|^2 \\ &= \left\| \pi_{[l_k:r_k]}(\mathbf{t}) - \mathbf{w}_k^{(\pi)} \right\|^2 + \left\| \pi_{[0:l_k]}(\mathbf{t}) - \sum_{i=1}^{k-1} \mathbf{w}_i \right\|^2 \\ &= \dots = \sum_{i=1}^k \left\| \pi_{[l_i:r_i]}(\mathbf{t}_i) - \mathbf{w}_i^{(\pi)} \right\|^2 = \sum_{i=1}^k \text{GH}^2(\mathbf{B}_{\pi[l_i, r_i]}) = \text{Dist}(\Psi). \square \end{aligned}$$

6.2 Properties of NColattice Blocksize Strategy

This section investigates the structural properties of blocksize strategies in the NColattice framework.

Specifically, Theorem 3 shows that, given a fixed sequence of blocksize parameters $\{\beta_1, \dots, \beta_k\}$, arranging the blocks in non-increasing order yields a blocksize strategy that minimizes the approximate distance to the target vector among all permutations of the sequence. This result is first established for the base case $k = 2$ in Lemma 1, and then generalized to arbitrary $k \geq 2$ in Theorem 3. The proofs of both Lemma 1 and Theorem 3 are provided in Appendix A.

Next, we propose Lemma 2 (see its proof in Appendix A) to compare the time cost and output distance of the strategies $\llbracket x, y \rrbracket$ and $\llbracket x - 1, y + 1 \rrbracket$, and extend this property to arbitrary strategies. We then present Theorem 4, which proves that the averaged strategy achieves both maximal distance and minimal time complexity; its proof is also included in Appendix A. Finally, Theorem 5 establishes that NCSearch returns the strategy with the minimal time cost under a given upper bound on the Euclidean norm distance.

Lemma 1. *Let $x, y \in \mathbb{Z}^*$ such that $x + y = d$ and $x \leq y$. Given the NColattice blocksize strategies $\Psi_1 = \llbracket x, y \rrbracket$ and $\Psi_2 = \llbracket y, x \rrbracket$, we have $\text{Dist}(\Psi_1) \geq \text{Dist}(\Psi_2)$.*

Theorem 3. *Let $\beta_i \in \mathbb{Z}^*$ for each $i \in \{1, 2, \dots, k\}$ such that*

$$\sum_{i=1}^k \beta_i = d \quad \text{and} \quad \beta_i \geq \beta_{i+1} \text{ for all } 1 \leq i \leq k-1.$$

Define the set of blocksize strategies as

$$\text{BS} := \{ \llbracket \beta_{\sigma_1}, \beta_{\sigma_2}, \dots, \beta_{\sigma_k} \rrbracket : (\sigma_1, \sigma_2, \dots, \sigma_k) \text{ is a permutation of } (1, 2, \dots, k) \}.$$

Let $\Psi_0 = \llbracket \beta_1, \beta_2, \dots, \beta_k \rrbracket \in \text{BS}$ denote the ordered blocksize strategy. Then, for all $\Psi \in \text{BS}$, the following inequality holds:

$$\text{Dist}(\Psi_0) \leq \text{Dist}(\Psi), \tag{7}$$

with equality if and only if $\Psi = \Psi_0$.

Lemma 2. *Let $x \in \mathbb{Z}^*$ and d be the dimension of the lattice. Then we have*

1. $(1) \llbracket x, y \rrbracket <_\gamma \llbracket x - 1, y + 1 \rrbracket, \forall x \geq 2; (2) T_{\text{NC}}(\llbracket x, y \rrbracket) > T_{\text{NC}}(\llbracket x - 1, y + 1 \rrbracket), \forall d > x > \lceil d/2 \rceil.$
2. $(1) \llbracket x, d - x \rrbracket <_\gamma \llbracket x', d - x' \rrbracket, \forall x > x' \geq 2; (2) T_{\text{NC}}(\llbracket x, d - x \rrbracket) > T_{\text{NC}}(\llbracket x', d - x' \rrbracket), \lceil d/2 \rceil \leq x' < x.$
3. *In particular, $\forall x > \lceil d/2 \rceil$, we have (1) $\llbracket x, d - x \rrbracket <_\gamma \llbracket \lceil d/2 \rceil, \lfloor d/2 \rfloor \rrbracket$, (2) $T_{\text{NC}}(\llbracket x, d - x \rrbracket) > T_{\text{NC}}(\llbracket \lceil d/2 \rceil, \lfloor d/2 \rfloor \rrbracket).$*
4. *For a k -size non-increasing strategy $\Psi = \llbracket \beta_1, \dots, \beta_{z_1}, \dots, \beta_{z_2}, \dots, \beta_k \rrbracket$ and Ψ' , where $\Psi'[z_1] = \beta_{z_1} - 1, \Psi'[z_2] = \beta_{z_2} + 1$ and Ψ' remains non-increasing, $\forall 1 \leq z_1 < z_2 \leq k$. Then we have $\Psi <_\gamma \Psi'$ and $T_{\text{NC}}(\Psi) > T_{\text{NC}}(\Psi').$*

Theorem 4. *Among all non-increasing k -size NColattice strategies for the same lattice with dimension d , the averaged strategy $\text{AvgStrategy}(d, k)$ achieves the maximal distance bound with minimal time complexity.*

Theorem 5. *NCSSearch outputs the minimum time-cost strategy within a given upper bound on the Euclidean norm distance.*

Proof. For the case $k_0 = 2$. Let $\Psi_{\min} = \llbracket x, d - x \rrbracket$ with $x \geq \lceil d/2 \rceil$. Suppose $\exists \Psi' = \llbracket x', d - x' \rrbracket$ with $T_{\text{NC}}(\Psi') < T_{\text{NC}}(\Psi_{\min})$ and $\text{Dist}(\Psi') < \gamma \text{GH}(\mathcal{L})$. By Lemma 2, $T_{\text{NC}}(\Psi') < T_{\text{NC}}(\Psi_{\min})$ implies $\lceil d/2 \rceil \leq x' < x$. It infers that $\text{Dist}(\Psi') < \text{Dist}(\Psi_{\min})$. This would make Ψ' replace $\Psi_{\min}$ by line 12 in SSearchLoop , contradicting $\Psi_{\min}$'s minimality.

For the case $k_0 > 2$. Suppose $\exists \Psi' = \llbracket \beta'_1, \beta'_2, \dots, \beta'_k \rrbracket$ with $T_{\text{NC}}(\Psi') < T_{\text{NC}}(\Psi_{\min})$ and $\text{Dist}(\Psi') < \gamma \text{GH}(\mathcal{L})$. From Theorem 4, we have $k \geq k_0$. If Ψ' is the minimum k -size strategy within distance bound. If $k \neq \sharp \Psi_{\min}$, $\Psi_{\min}$ would be replaced with Ψ' by line 12 in NCSSearch , contradicting $\Psi_{\min}$'s minimality.

If $k = \sharp \Psi_{\min}$, consider the element difference between Ψ' and $\Psi_{\min}$. If $\beta'_1 \neq \Psi_{\min}[1]$. During the while loop in SSearchLoop , $\Psi_{\min}$ would be replaced with Ψ' by line 13 in SSearchLoop , contradicting $\Psi_{\min}$'s minimality. Similarly, it can be shown that if the values of β'_i differ for $k \geq i > 1$, a contradiction also arises. $\square$

7 Strategy-Optimized Nearest-Colattice Estimator in Two-step Mode

In this section, we develop two variants of approximate CVP estimators and their batch-CVP variants based on the Nearest-Colattice framework, specifically optimized for the Euclidean and infinity norm cases respectively.

In Sec. 7.1, we first establish our computational model and formalize the core CVP and SVP algorithms that will serve as the foundation for our estimators. This includes precise definitions of the time complexity models and approximation guarantees for these base algorithms.

Building on these foundations, we present in Sec. 7.2 our first estimator designed for the Euclidean norm case. The construction includes (1) The complete algorithmic framework for both CVP_γ and batch-CVP $_\gamma$; (2) Complexity analysis under our computational model.

Sec. 7.3 then describes the methodological modifications required to transform the Euclidean norm estimator into the infinity norm case.

7.1 Cost Model

In our estimation framework, we instantiate the SVP oracle for BKZ using the BDGL sieve algorithm [16], while implementing the CVP oracle through the randomized iterative slicer approach [1]. These represent the currently most efficient practical algorithms for solving SVP and CVP problems respectively.

The randomized iterative slicer algorithm traditionally relies on repeated iterations to find an exact closest vector. However, in our setting, since each

sub-block's output does not require the exact nearest vector, we can relax this condition. Specifically, we replace the randomized slicer with a single iterative slicer, which modifies the distance guarantee for each sub-block from $\leq \text{GH}(\mathcal{L})$ to $\leq \gamma_1 \cdot \text{GH}(\mathcal{L})$, where $\gamma_1 = \frac{\alpha^2}{2\sqrt{\alpha^2-1}} \in [1, 2)$ with $\alpha \in (\sqrt{6} - \sqrt{2}, \sqrt{2}]$.

Crucially, this relaxation does not significantly affect the quality of the approximate closest vector found for the full colattice. Therefore, we can effectively treat the iterative slicer as a CVP_{γ_1} oracle with $\gamma_1 = \frac{\alpha^2}{2\sqrt{\alpha^2-1}}$. An important observation is that scaling the distance bound in each sub-block to $\gamma \cdot \text{GH}(\mathcal{L})$ preserves the selection rules of our strategy optimization algorithm. Consequently, in the subsequent estimator design, we employ a CVP_{γ_1} oracle in place of an exact CVP oracle.

We categorize the estimation into two types: theoretical estimation (denoted as `naive_classical` cost model) and gate count estimation based on AGPS20 [17] and MATZOV22 [18] cost model.

Theoretical Estimation. From Eq. (4) in [1], the theoretical cost of one iterative slicer call is given by

$$T_{\text{iterative_slicer}}(\alpha, d) = \left(\alpha \cdot \sqrt{1 - \frac{2(1 - 1/\alpha^2)}{1 + \sqrt{1 - 1/\alpha^2}}} \right)^{-d}, \quad (8)$$

which increases monotonically with α . Conversely, the approximation factor γ_1 decreases with α . The choice $\alpha = \sqrt{4/3}$ represents a critical trade-off: while the iterative slicer with fewer short vectors can decrease the time cost $T_{\text{iterative_slicer}}$ with larger approximate factor γ_1 , but the sieve algorithm must sieve the lattice vectors up to the $(\frac{4}{3})^{d/2}$ -th shortest vector to guarantee the shortness of each lattice vector. So we choose $\alpha = \sqrt{4/3}$ as a trade-off between computational complexity and the approximate factor γ_1 in the iterative slicer tour.

In our theoretical complexity estimation framework, we establish the following parameters:

- **SVP coefficient:** We set $c_1 = 0.292$ following the current best known asymptotic complexity for lattice sieving algorithms [16]. The time cost of each SVP call is $T_{\text{SVP}} = 2^{0.292d}$.
- **CVP oracle complexity:** The time complexity of the CVP oracle is given by:

$$T_{\text{CVP}}(d) = 2^{c \cdot d} = 2^{c_1 d} + 2^{c_2 d}. \quad (9)$$

where:

- The term $2^{c_1 d}$ accounts for the sieving preprocessing cost (with $c_1 = 0.292$).
- The term $2^{c_2 d}$ represents the theoretical complexity of the iterative slicer algorithm with $c_2 = \log_2 \frac{3}{2\sqrt{2}} \approx 0.08496$. c_2 is obtained by setting $\alpha = \sqrt{4/3}$ in Eq. (8).

Gate Count Estimation. For SVP gate count estimation, we use the AGPS20 and MATZOV22 gate in the lattice estimator [19], where AGPS20 is proposed in [17] and MATZOV22 is an improvement over [17].

We analyze the CVP gate count estimation following [17], which treats the **popcount** operation as a fundamental component. In the iterative slicer (Alg. 1, line 4), each query requires a nearest neighbor search (NNS), and the total time complexity equals the product of a single NNS cost and the optimal number of arbitrary-step paths per iterative slicer tour. The concrete analysis is as follows:

Nearest Neighbor Search. The **popcount** operation serves as a fundamental component in Nearest Neighbor Search (NNS) algorithms. Following the implementation proposed by [17], it provides the computational costs of **popcount** for both classical and quantum computers.

- Let T_{pop} denote the time cost for one **popcount** operation. We adopt both classical and quantum gate counts for T_{pop} as proposed in [17].
- Let T_{dot} represent the inner product computation cost for vectors passing the **popcount** filter.
- Let p be the probability that a vector pair passes the **popcount** filter.
- Let L be the preprocessed sieving set generated from lattice basis $\mathbf{B}$ and $N = |L|$ denote the size of the sieving set.

The total computational cost for one NNS query to find the nearest neighbor of target vector $\mathbf{t}$ is given by $T_{\text{NNS}} = (p \cdot T_{\text{dot}} + T_{\text{pop}}) \cdot N$. In the **popcount** implementation described in [17], the T_{dot} term is omitted from the complexity analysis. This simplification yields the final NNS process complexity:

$$T_{\text{NNS}} = T_{\text{pop}} \cdot N. \quad (10)$$

Iterative Slicer. In the iterative slicer algorithm, NNS is called to identify the approximate vector $\mathbf{w} \in L$ to a target vector $\mathbf{t}$. We estimate the cost of the iterative slicer by Claim 1, which is based on Claim 3 in [1] and we add the cost of sieving preprocess for one iterative slicer time cost. The size of the sieve set determines both the time cost of sieving and the approximation ratio of the output from the iterative slicer tour.

Claim 1 (A modification of Claim 3 in [1]) *Given a list L of the $N = \alpha^d \leq (4/3)^{d/2}$ shortest lattice vectors as input, we can solve a single CVP_{γ_1} instance with $\gamma_1 = \frac{\alpha^2}{2\sqrt{\alpha^2-1}}$ and the following complexities:*

$$T_{\text{iter_slicer}}(\alpha, d) = N \cdot t \cdot T_{\text{pop}} = \left(\alpha \cdot \sqrt{1 - \frac{2 \cdot (1 - 1/\alpha^2)}{1 + \sqrt{1 - 1/\alpha^2}}} \right)^{-d} \cdot T_{\text{pop}}, \quad (11)$$

where t is the optimal number of arbitrary-step paths for each iterative slicer tour.

When considering the batch-CVP problem involving multiple target vectors with a single lattice basis, the sieving preprocessing phase only needs to be performed once. This approach significantly reduces the average solving time per individual CVP instance, leading to Theorem 6.

Theorem 6 (Time Complexity for batch-CVP under Infinity Norm Bound, extended version of Heuristic Claim 6 in [1]). *Let L be a set of α^d produced by a d -dimensional sieving algorithm. Given a target vector set $\{\mathbf{t}_i\}_m$ with $m \leq \max\{1, \alpha^{\beta_1} \cdot p_{B, \alpha, \beta_1}\}$. For a vector $\mathbf{w}$ produced by an *RandNColattice* tour with strategy $\Psi = \llbracket \beta_1, \beta_2, \dots, \beta_k \rrbracket$, where β_1 is computed via an approximate CVP $_{\gamma_1}$ call. The time cost for the *RandNColattice* tour to solve per CVPP instance in the batch-CVP instances is*

$$T_{\text{per_batch_CVP}} = \frac{1}{m} \sum_{i=1}^k T_{\text{sieve}}(\beta_i) + T_{\text{iter_slicer}}(\alpha, \beta_1) + \sum_{i=2}^k T_{\text{iter_slicer}}(\alpha, \beta_i), \quad (12)$$

*the total time cost for the *RandNColattice* tour to solve the batch-CVP instances is*

$$T_{\text{batch_CVP}} = m \cdot T_{\text{per_batch_CVP}}. \quad (13)$$

7.2 Improved Nearest-Colattice Estimator

In this part, we present an Estimator (Alg. 9) for Approximate CVP based on *TwoStepNColattice* (Alg. 5) and its strategy optimization method *NCSSearch* (Alg. 7). Given a lattice volume V , dimension d , and a target distance ratio γ , the procedure begins by initializing $T_{\min}$ as the exact CVP time cost $T_{\text{NC}}(d)$. It then iteratively explores reduced blocksizes β ranging from 2 to d . For each β , the lattice basis is predicted by BKZ 2.0 simulator [14] after a series of BKZ tours from blocksize 2 up to β . Based on the predicted Gram-Schmidt norms, find a minimum time-cost strategy Ψ by *NCSSearch* that meets the target distance ratio γ . The total time-cost T is then computed as the sum of the preprocessing time $T_{\text{ProBKZ}}(\beta, d) = \frac{1}{1-2^{c_1}} \cdot (d - \beta) \cdot 2^{c_1\beta}$, which accounts for the cost of performing the progressive BKZ tours from 2 up to β , and the Approximate CVP time $T_{\text{NC}}(\Psi) = \sum_{\beta \in \Psi} 2^{c\beta}$ for the reduced lattice. If this cost is lower than the current minimum, $T_{\min}$ is updated accordingly. After evaluating all values of β , the algorithm returns the smallest observed time-cost.

If we consider the dimension-for-free technique [15], then $T_{\text{ProBKZ}}(\beta, d) = \frac{1}{1-2^{c_1}} \cdot (d - \beta) \cdot 2^{c_1(\beta - \text{d4f}(\beta))}$ and $\Psi_{\text{NC}}(\Psi) = \sum_{\beta \in \Psi} 2^{c(\beta - \text{d4f}(\beta))}$, where $\text{d4f} = \left\lceil \frac{\beta \cdot \ln(\frac{4}{3})}{\ln(\frac{\beta}{2\pi e})} \right\rceil$ is the dimension-for-free function proposed in [15].

7.3 Improved Nearest-Colattice Estimator in Infinity Norm

Since many lattice-based signature schemes (e.g. Dilithium) utilize the infinity norm for distance bounding, we propose a randomized Nearest-Colattice Algorithm that generates multiple approximate closest vectors and analyze the success probability of solving CVP $_{\gamma}$ in the infinity norm in this section.

input : Volume of lattice V , dimension d , a target distance ratio $\gamma \in \mathbb{R}$;
output: The minimum time-cost $T_{\min}$;
1 Function SopNCEstimator(V, d, γ):
2 $T_{\min} \leftarrow T_{\text{NC}}(\lfloor d \rfloor)$;
3 **for** β **from** 2 **to** d **do**
4 $rr \leftarrow$ GS-norms generated by $\delta(\beta)$ and V based on GSA;
5 $\Psi \leftarrow \text{NCSSearch}(rr, \gamma)$; $T \leftarrow T_{\text{ProBKZ}}(\beta, d) + T_{\text{NC}}(\Psi)$;
6 **if** $T < T_{\min}$ **then**
7 $T_{\min} \leftarrow T$;
8 **return** $T_{\min}$;

Algorithm 9: Strategy-Optimized TwoStepNColattice Estimator

Amount of vectors within the Approximate Voronoi Cell. From Corollary 4.1 in [9], when we have a short vector list L with $|L| = \alpha^d$ with $\alpha \leq \sqrt{2}$, then the volume of the approximate Voronoi is $\text{Vol}(\mathcal{V}_L) \approx \left(\frac{\alpha^4}{4\alpha^2-4}\right)^{d/2} \text{Vol}(\mathcal{V})$. In the proof of Theorem 3.2 in [9], it regards $\mathcal{V}_L$ as a hyper ball, we follow the same assumption. Then, the Euclidean norm distance output by one iterative slicer becomes $\frac{\alpha^2}{2\sqrt{\alpha^2-1}} \text{GH}(\mathcal{L})$, the amount of potential solutions during one iterative slicer call is

$$M = \frac{\text{Vol}(\mathcal{V}_L)}{\text{Vol}(\mathcal{V})} = \frac{\text{Vol}\left(\mathcal{B}^d_{\sqrt{\frac{\alpha^4}{4\alpha^2-4}} \text{GH}(\mathcal{L})}\right)}{\text{Vol}(\mathcal{B}^d_{\text{GH}(\mathcal{L})})} = \left(\frac{\alpha^4}{4\alpha^2-4}\right)^{d/2}. \quad (14)$$

Success Probability in Infinity Norm. Consider the relation between the infinity norm bound and Euclidean norm bound, especially on the aspect of CVP. From Theorem 6.2 in [11] we have Claim 2.

Claim 2 *Let L be a set of α^d (with $\alpha \in (\sqrt{6} - \sqrt{2}, \sqrt{2}]$) produced by a d -dimensional sieving algorithm. For a vector $\mathbf{w}$ produced by an iterative slicer tour such that $\|\mathbf{w} - \mathbf{t}\| \leq \gamma \cdot \text{GH}(\mathcal{L})$, the probability that $\|\mathbf{w} - \mathbf{t}\|_{\infty} \leq B$ is*

$$p_{B,\alpha,d} = \Pr(\|\mathbf{v}\|_{\infty} \leq B) = \left(1 - 2\Phi\left(-\frac{\sqrt{d}B}{\gamma \cdot \text{GH}(\mathcal{L})}\right)\right)^d, \quad (15)$$

where Φ is the cumulated distribution function for the standard Gaussian $\mathcal{N}(0, 1)$ and $\gamma = \frac{\alpha^2}{2\sqrt{\alpha^2-1}}$.

To improve the likelihood of finding approximate closest vectors to a given target vector in infinity norm using the randomized iterative slicer, we should adjust the repeating times according to Eq. (15). To guarantee that the solution can be found by increasing the number of repetitions, the inequality $M \geq \frac{1}{p_{B,\alpha,d}}$ must be satisfied. Then we have Theorem 7.

Theorem 7 (Time Complexity for Single CVP under Infinity Norm Bound). *Let L be a set of α^d produced by a d -dimensional sieving algorithm. For*

a vector $\mathbf{w}$ produced by an *RandNColattice* tour with strategy $\Psi = \llbracket \beta_1, \beta_2, \dots, \beta_k \rrbracket$ and consider iterative slicer as its CVP $_{\gamma_1}$ call with $\gamma_1 = \frac{\alpha^2}{2\sqrt{\alpha^2-1}}$. Then we can find the solution such that $\|\mathbf{w}\|_\infty \leq B$ with probability

$$p = \min\left\{1, \left(\frac{\alpha^4}{4\alpha^2-4}\right)^{d/2} \cdot \left(1 - 2\Phi\left(-\frac{\sqrt{d}B}{\text{Dist}'(\Psi)}\right)\right)^d\right\}, \quad (16)$$

where $\text{Dist}'(\Psi) = \sum_{i=1}^k \gamma_1^2 \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}, d_i]})$ and $d_i = \sum_{j=1}^i \beta_j$. If $p = 1$, the time cost of an *RandNColattice* tour is

$$T_{\text{RandNColattice}} = \sum_{i=1}^k T_{\text{sieve}}(\beta_i) + \frac{1}{p_{B, \alpha, \beta_1}} \cdot T_{\text{iter_slicer}}(\alpha, \beta_1) + \sum_{i=2}^k T_{\text{iter_slicer}}(\alpha, \beta_i) \quad (17)$$

with $l_W = \frac{1}{p_{B, \alpha, \beta_1}}$; Otherwise,

$$T_{\text{RandNColattice}} = \sum_{i=1}^k T_{\text{sieve}}(\beta_i) + M \cdot T_{\text{iter_slicer}}(\alpha, \beta_1) + \sum_{i=2}^k T_{\text{iter_slicer}}(\alpha, \beta_i) \quad (18)$$

with $l_W = \left(\frac{\alpha^4}{4\alpha^2-4}\right)^{d/2}$.

8 Security Estimation for Post-Quantum Schemes

In this section, we evaluate the post-quantum signature schemes Dilithium, one-more-ISIS, and HuFu against forgery attacks using the estimator and time-cost model introduced in Sec. 7. The security of all three schemes under such attacks relies on the hardness of the ISIS problem or its variants. Therefore, to facilitate the subsequent security analysis, we first introduce a reduction from the ISIS problem to an approximate CVP problem. This reduction allows us to leverage CVP-based attack models for estimating the concrete security of these schemes. According to the NIST PQC security criteria [20], the required security levels I–V correspond to 143, 146, 207, 210, and 272 classical gate counts, respectively.

We define the ISIS problem as follows:

Definition 11 (ISIS). Let $\mathbf{A} \leftarrow_{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{t} \leftarrow_{\$} \mathbb{Z}_q^n$, the goal of solving $\text{ISIS}_{n,m,B}$ is to find short vectors $\mathbf{z} \in \mathbb{Z}_q^m$, $\mathbf{u} \in \mathbb{Z}_q^n$ such that $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{t} \bmod q$ and $\left\| \begin{pmatrix} \mathbf{z} \\ \mathbf{u} \end{pmatrix} \right\| \leq B$.

The reduction from ISIS to approximate CVP has been discussed in [11]. Here, we briefly review it. Let the lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$ with column-wise basis $\mathbf{B} = \begin{pmatrix} q\mathbf{I} & \mathbf{A} \\ \mathbf{O} & \mathbf{I} \end{pmatrix}$ and the target vector of the approximate CVP is $\begin{pmatrix} \mathbf{t} \\ \mathbf{0} \end{pmatrix}$ and we want to find the approximate close vector $\begin{pmatrix} \mathbf{A}\mathbf{z} \\ \mathbf{z} \end{pmatrix} \in \mathcal{L}(\mathbf{B})$.

8.1 Security Estimation for Dilithium

In Dilithium, its signature forgery attack relies on the hardness of the $\text{ISIS}^{(\infty)}$ problem, which is the approximate CVP considering the infinity norm. We define the $\text{ISIS}^{(\infty)}$ problem as follows:

Definition 12 ($\text{ISIS}^{(\infty)}$). Let $\mathbf{A} \leftarrow_{\$} \mathbb{Z}_q^{m \times n}$, $\mathbf{t} \leftarrow_{\$} \mathbb{Z}_q^n$, the goal of solving $\text{ISIS}_{n,m,B}^{(\infty)}$ is to find short vectors $\mathbf{z} \in \mathbb{Z}_q^m$, $\mathbf{u} \in \mathbb{Z}_q^n$, $\|\mathbf{z}\|_\infty \leq B_{\mathbf{z}}$, $\|\mathbf{u}\|_\infty \leq B_{\mathbf{u}}$ such that $\mathbf{A}\mathbf{z} + \mathbf{u} = \mathbf{t} \pmod{q}$.

The Dilithium specification adopts a unified bound $B = \max\{B_{\mathbf{u}}, B_{\mathbf{z}}\}$, where $B_{\mathbf{u}}$ and $B_{\mathbf{z}}$ are defined in Equation (7) of the Dilithium paper [2]. To ensure consistency with this specification, our analysis in this paper adopts the same approach.

To make sure the solution is also within the ℓ_∞ -norm bound, we consider the computational complexity of NColattice from Theorem 7.

If consider $\mathbf{B}_\mu = \begin{pmatrix} \mu q \mathbf{I} & \mu \mathbf{A} \\ \mathbf{O} & \mathbf{I} \end{pmatrix}$ as [11], then $B = B_{\mathbf{z}}$ and $\text{Vol} = (\mu q)^n$ with $\mu = B_{\mathbf{z}}/B_{\mathbf{u}}$. We give the security estimation in both cases as Table 7 in Appendix B and Table 3. The estimation results in both tables indicate that Dilithium at security categories 3 and 5 fails to meet the required security levels under our attack model, with gate counts falling more than $20 \log_2(\text{gates})$ below the corresponding thresholds.

8.2 Security Estimation for HuFu

The signature forgery attack on HuFu can be formulated as solving an ISIS problem. In the signature scheme, let $(\mathbf{A} = [\mathbf{I}, \hat{\mathbf{A}}, \mathbf{B}], \mathbf{u}) \in \mathbb{Z}_Q^{m \times (2m+n)} \times \mathbb{Z}_Q^m$, where $\hat{\mathbf{A}} \in \mathbb{Z}_Q^{m \times n}$ and $\mathbf{B} \in \mathbb{Z}_Q^{m \times m}$. The signature $\mathbf{x} = (\mathbf{x}_0 + \mathbf{e}, \mathbf{x}_1, \mathbf{x}_2)$ satisfies the equation

$$\mathbf{x}_0 + \mathbf{e} + \hat{\mathbf{A}}\mathbf{x}_1 + \mathbf{B}\mathbf{x}_2 = \mathbf{u} \pmod{Q}.$$

The signature verification process involves computing $\mathbf{x}_0 + \mathbf{e} = \mathbf{u} - \hat{\mathbf{A}}\mathbf{x}_1 - \mathbf{B}\mathbf{x}_2 \pmod{Q}$ and checking $\|(\mathbf{x}_0 + \mathbf{e}, \mathbf{x}_1, \mathbf{x}_2)\| \leq B$.

For forgery attack, we should find another $\mathbf{x}$ to pass the verification process. This problem can be regarded as an $\text{ISIS}_{m,m+n,B}$ problem. HuFu paper [4] reduced it to an approximate CVP and estimate its hardness by the Nearest-Colattice algorithm [10]. The algorithm employs BKZ- β and a Nearest-Colattice with blocksize β to solve the problem. It consider the attack by finding the optimal (β, k) that can minimize the right side of the inequality:

$$B \geq \min_{k \leq m+n} (\delta_\beta^{2m+n-k} Q^{\frac{m}{2m+n-k}}).$$

Here, $\delta_\beta^{2m+n-k} Q^{\frac{m}{2m+n-k}}$ represents the estimated approximate closest vector in a $(2m+n-k)$ -dimensional lattice.

However, the estimation in HuFu paper only considers the time cost of the attack as one SVP oracle with block size β , which is a conservative estimation. From Table 2 presented in HuFu paper, we re-estimate its hardness of forgery in Table 4.

Table 3. Security Estimation for Dilithium: `classical_agps20` and `classical_matzov22` Cost Model.

Estimator	Parameter	classical_agps20			classical_matzov22		
		Lv2	Lv3	Lv5	Lv2	Lv3	Lv5
Required Level	$\log_2(\text{gates})$	146	207	272	146	207	272
$B = \max\{B_u, B_z\}$		350209	724481	769537	350209	724481	769537
Restricted SVP	β	436 (398)	661 (609)	936 (869)	-	-	-
	d_{svp}	447 (436)	-	-	-	-	-
	$\log_2(\text{gates})$	159.9	223.0	300.3	-	-	-
NC Strategy	β	441 (402)	666 (614)	946 (879)	441 (402)	667 (615)	947 (880)
	β_1	428	647	912	432	648	913
	$-\log_2(p_{B,\alpha,\beta_1})$	99.6	141.4	197.0	89.2	135.0	190.2
	$\log_2(\text{gates})$	159.4	223.5	302.9	152.9	216.5	295.3
Optimized Strategy	β	441 (402)	666 (614)	945 (878)	441 (402)	667 (615)	946 (879)
	β_1	427	646	914	431	647	915
	$-\log_2(p_{B,\alpha,\beta_1})$	101.1	143.0	198.9	95.6	136.5	192.1
	$\log_2(\text{gates})$	159.3	223.4	302.7	152.7	216.4	295.1
$B = B_z$		130994	524092	524168	130994	524092	524168
Restricted SVP	β	478 (437)	677 (625)	963 (895))	-	-	-
	d_{svp}	485 (475)	-	-	-	-	-
	$\log_2(\text{gates})$	171.5	227.7	307.9	-	-	-
NC Strategy	β	482 (441)	683 (630)	973 (904)	482 (441)	684 (631)	974 (905)
	β_1	470	664	940	474	665	941
	$-\log_2(p_{B,\alpha,\beta_1})$	109.8	144.4	202.9	95.4	138.2	196.4
	$\log_2(\text{gates})$	171.1	228.3	310.6	164.6	221.3	302.9
Optimized Strategy	β	481 (440)	683 (630)	972 (903)	482 (441)	683 (630)	973 (904)
	β_1	472	663	942	473	667	943
	$-\log_2(p_{B,\alpha,\beta_1})$	107.0	150.9	204.6	101.2	135.4	193.2
	$\log_2(\text{gates})$	170.9	228.0	310.3	164.5	221.1	302.7

d_{svp} : sieve dimension.

β : BKZ block size.

β_1 : The first block of Ψ for `RandNColattice`.

(·): dimension accounting for “dimensions for free” [15] in BKZ preprocess.

Time cost of Restricted SVP ignores the cost of slicer tours in its estimation.

8.3 Security Estimation for one-more-ISIS Blind Signature

The one-more-ISIS problem serves as the foundation for the security of the blind signature scheme [3]. The blind signature is constructed by the following steps: (1) The user samples $\mathbf{h}' \in \mathcal{R}_{512}$ and short vectors $(\mathbf{x}_1, \mathbf{x}_2)$ with $\|(\mathbf{x}_1, \mathbf{x}_2)\|_\infty \leq 2$, then computes $\mathbf{t} = \mathbf{h}'\mathbf{x}_1 + \mathbf{x}_2 + H(\mu) \bmod q_F$ where $H(\mu)$ is the hash of the message μ . (2) The value $\mathbf{t}$ is sent to the signer, who responds by generating a short preimage $\mathbf{y} = (\mathbf{y}_1 \parallel \mathbf{y}_2)$ through the `SamplePre` routine such that $\mathbf{h} \cdot \mathbf{y}_1 + \mathbf{y}_2 = \mathbf{t} \bmod q$, with $\|\mathbf{y}\|$ bounded by B . $\mathbf{y} = (\mathbf{y}_1 \parallel \mathbf{y}_2)$ constitutes the signature in this blind signature scheme.

The adversary can forge a valid signature by generating $\mathbf{h}'$ by $\mathbf{h}' = \mathbf{x}'_1 \cdot \mathbf{h} + \mathbf{x}'_2 \bmod q$, where $\|\mathbf{x}'_1\|_\infty \leq 2$ and $\|\mathbf{x}'_2\|_\infty \leq 2$. This enables the construction of an alternative solution $\mathbf{z} = (\mathbf{y}_1 - \mathbf{x}_1\mathbf{x}'_1 \parallel \mathbf{y}_2 - \mathbf{x}_1\mathbf{x}'_2 - \mathbf{x}_2)$ that satisfies $(\mathbf{h} \parallel 1) \cdot \mathbf{z} = H(\mu) \bmod q$. The forgery succeeds if the norm of $\mathbf{z}$ satisfies $\|\mathbf{z}\| \leq B$.² If transform into the one-more-ISIS problem, the parameters set as follows: $n = m = 512$, $q = 7213$, $B = 4469.52$. It infers that $\gamma = 6.8$. The required security is 128 bits. The security estimation result is as Tab. 5.

8.4 Consider Multiple Queries in Signature Forgery

Since in the forgery attack model, the adversary can typically make multiple signing queries when the signer uses the same secret key. Assuming the number of queries is bounded by $\mathcal{Q} = 2^{32}$, we can re-evaluate the security levels of Dilithium, HuFu, and the one-more-ISIS problem through the lens of batch-CVP complexity.

Using Theorem 6 for batch processing estimation, we derive updated security bounds as presented in Table 6. This analysis accounts for the amortized cost reduction achieved by sharing lattice preprocessing across multiple CVP instances. Table 6 shows that under the batch-CVP attack model with 2^{32} queries, none of the evaluated schemes—Dilithium (with different bounds), HuFu, or one-more-ISIS achieve the required security gate counts for their respective levels. This indicates that all these schemes fall short of meeting their claimed security levels under these attack assumptions.

² The original scheme analyzed a more relax condition $\|\mathbf{z}\| \leq \sqrt{B^2 + 24 \cdot 512 \cdot 4 + 22 \cdot 512 \cdot 2}$, which cannot make the forgery succeed.

Table 4. Security Estimation for HuFu.

	Estimator	Parameter	Cost Model		
			classical_naive	classical_matzov22	
			no d4f	Consider d4f	
Lv1	Required Level	$\log_2(\text{gates})$	128	128	143
	HuFu paper	β bits	438 128	- -	- -
	NC Strategy	β	456	470 (430)	471 (431)
		β_1	494	465	463
		$\log_2(\text{gates})$	146.6	138.7	160.9
	Optimized Strategy	β	458	469 (429)	469 (429)
		β_1	490	467	467
		$\log_2(\text{gates})$	146.3	138.2	160.4
Lv3	Required Level	$\log_2(\text{gates})$	192	192	207
	HuFu paper	β bits	659 192	- -	- -
	NC Strategy	β	713	733 (677)	735 (679)
		β_1	754	718	715
		$\log_2(\text{gates})$	222.1	211.6	234.8
	Optimized Strategy	β	713	733 (677)	733 (677)
		β_1	754	718	718
		$\log_2(\text{gates})$	221.7	211.2	234.4
Lv5	Required Level	$\log_2(\text{gates})$	256	256	272
	HuFu paper	β bits	883 258	- -	- -
	NC Strategy	β	917	940 (873)	940 (873)
		β_1	955	914	914
		$\log_2(\text{gates})$	281.5	269.0	293.1
	Optimized Strategy	β	915	940 (873)	940 (873)
		β_1	958	914	914
		$\log_2(\text{gates})$	281.1	268.6	292.7

Table 5. Security Estimation for one-more-ISIS Blind Signature.

Estimator	Parameter	Cost Model		
		<code>classical_naive</code>	<code>classical_matzov22</code>	
		no d4f	Consider d4f	
Required Level	$\log_2(\text{gates})$	128	128	143
one-more-ISIS paper	bits	109	-	-
NC Strategy	β	434	452 (413)	453 (414)
	β_1	471	450	449
	$\log_2(\text{gates})$	139.5	133.3	155.5
Optimized Strategy	β	433	452 (413)	452 (413)
	β_1	472	450	450
	$\log_2(\text{gates})$	139.0	132.9	155.0

Table 6. Security Estimation based on batch-CVP .

Scheme	Parameter	Lv1	Lv2	Lv3	Lv5
Required Level	$\log_2(\text{gates})$	143	146	207	272
Dilithium with $B = \max\{B_{\mathbf{u}}, B_{\mathbf{z}}\}$	β	-	446 (407)	672 (620)	952 (884)
	β_1	-	438	654	917
	$-\log_2(p_{B,\alpha,\beta_1})$	-	61.3	105.0	160.5
	$\log_2(\text{gates})$	-	122.4	186.0	264.5
Dilithium with $B = B_{\mathbf{z}}$	β	-	488 (447)	633 (583)	979 (910)
	β_1	-	479	836	946
	$-\log_2(p_{B,\alpha,\beta_1})$	-	70.8	112.0	165.9
	$\log_2(\text{gates})$	-	134.3	190.8	272.4
HuFu	β	471 (431)	-	735 (679)	940 (873)
	β_1	463	-	715	914
	$\log_2(\text{gates})$	128.9	-	202.8	261.1
one-more-ISIS	β	453 (414)	-	-	-
	β_1	449	-	-	-
	$\log_2(\text{gates})$	123.5	-	-	-

The security estimates in this table are based on the `classical_matzov22` cost model, the **d4f** technique for BKZ preprocess, an optimized `NColattice` strategy, and Eq. (12) as the per-call cost in batch-CVP.

References

1. L. Ducas, T. Laarhoven, and W. P. van Woerden, “The randomized slicer for cvpp: sharper, faster, smaller, batchier,” in *IACR International Conference on Public-Key Cryptography*, pp. 3–36, Springer, 2020.
2. Bai, Shi, L. Ducas, E. Kiltz, Lepoint, Tancrede, V. Lyubashevsky, P. Schwabe, G. Seiler, and D. Stehlé, “Dilithium,” 2021.
3. S. Agrawal, E. Kirshanova, D. Stehlé, and A. Yadav, “Practical, round-optimal lattice-based blind signatures,” in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pp. 39–53, 2022.
4. Y. Yu, H. Jia, L. Li, D. Ran, Z. Qiu, S. Zhang, X. Lin, and X. Wang, “Hufu: Hash-and-sign signatures from powerful gadgets,”
5. T. Prest, P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, “Falcon,” tech. rep., Technical report, National Institute of Standards and Technology, 2020.
6. D. Bundestag, “Antrag,” *Europäische Ausländer-, Asyl-und Zuwanderungspolitik transparent machen*, *Drucksachen*, vol. 15, no. 655, pp. 17–3, 2003.
7. N. Sommer, M. Feder, and O. Shalvi, “Finding the closest lattice point by iterative slicing,” *SIAM Journal on Discrete Mathematics*, vol. 23, no. 2, pp. 715–731, 2009.
8. E. Doulgerakis, T. Laarhoven, and B. de Weger, “Finding closest lattice vectors using approximate voronoi cells,” in *Post-Quantum Cryptography: 10th International Conference, PQCrypto 2019, Chongqing, China, May 8–10, 2019 Revised Selected Papers 10*, pp. 3–22, Springer, 2019.
9. T. Laarhoven, “Approximate voronoi cells for lattices, revisited,” *Journal of Mathematical Cryptology*, vol. 15, no. 1, pp. 60–71, 2020.
10. T. Espitau and P. Kirchner, “The nearest-colattice algorithm,” *arXiv preprint arXiv:2006.05660*, 2020.
11. G. Wang, W. Xia, and D. Gu, “Heuristic algorithm for solving restricted svp and its applications,” in *International Conference on Post-Quantum Cryptography*, pp. 119–152, Springer, 2025.
12. W. Xia, L. Wang, GengWang, D. Gu, and B. Wang, “Refined strategy for solving LWE in two-step mode.” *Cryptology ePrint Archive*, Paper 2022/1343, 2022.
13. P. Q. Nguyen, “Hermite’s constant and lattice algorithms,” in *The LLL Algorithm: Survey and Applications*, pp. 19–69, Springer, 2009.
14. Y. Chen and P. Q. Nguyen, “Bkz 2.0: Better lattice security estimates,” in *International Conference on the Theory and Application of Cryptology and Information Security*, pp. 1–20, Springer, 2011.
15. L. Ducas, “Shortest vector from lattice sieving: a few dimensions for free,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 125–145, Springer, 2018.
16. A. Becker, L. Ducas, N. Gama, and T. Laarhoven, “New directions in nearest neighbor searching with applications to lattice sieving,” in *Proceedings of the twenty-seventh annual ACM-SIAM symposium on Discrete algorithms*, pp. 10–24, SIAM, 2016.
17. M. R. Albrecht, V. Gheorghiu, E. W. Postlethwaite, and J. M. Schanck, “Estimating quantum speedups for lattice sieves,” in *Advances in Cryptology—ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II 26*, pp. 583–613, Springer, 2020.

18. MATZOV, “Report on the Security of LWE: Improved Dual Lattice Attack.” <https://doi.org/10.5281/zenodo.6412487>, Apr. 2022.
19. M. R. Albrecht, R. Player, and S. Scott, “On the concrete hardness of learning with errors,” *Journal of Mathematical Cryptology*, vol. 9, pp. 169–203, October 2015.
20. National Institute of Standards and Technology (NIST), “Security evaluation criteria.” NIST Post-Quantum Cryptography Standardization Project, 2023. Accessed: 2023-11-01.
21. M. R. Albrecht, L. Ducas, G. Herold, E. Kirshanova, E. W. Postlethwaite, and M. Stevens, “The general sieve kernel and new records in lattice reduction,” in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pp. 717–746, Springer, 2019.

A Some Proofs for the Correctness of NCSSearch

Lemma 1. Let $x, y \in \mathbb{Z}^*$ such that $x + y = d$ and $x \leq y$. Given the `NColattice` blocksize strategies $\Psi_1 = \llbracket x, y \rrbracket$ and $\Psi_2 = \llbracket y, x \rrbracket$, we have $\text{Dist}(\Psi_1) \geq \text{Dist}(\Psi_2)$.

Proof. From Theorem 2,

$$\begin{aligned} \text{Dist}(\Psi_1) &= \text{GH}^2(\mathcal{L}_{\pi[\cdot:x]}) + \text{GH}^2(\mathcal{L}_{\pi[x:\cdot]}) \\ &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-1}{d-1})}, \end{aligned} \quad (19)$$

$$\text{Dist}(\Psi_2) = \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{y-1}{d-1})} + \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2y+x-1}{d-1})}.$$

$$\begin{aligned} &\text{Dist}(\Psi_1) - \text{Dist}(\Psi_2) \\ &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-1}{d-1})} - \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{y-1}{d-1})} - \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2y+x-1}{d-1})} \\ &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{d-x}{2\pi e} \cdot \delta^{-\frac{2dx}{d-1}} - \frac{d-x}{2\pi e} \cdot \delta^{\frac{2dx}{d-1}} - \frac{x}{2\pi e} \cdot \delta^{-2d \cdot \frac{d-x}{d-1}} \\ &= \frac{x}{2\pi e} \cdot \delta^{\frac{2d^2}{d-1} - \frac{2dx}{d-1}} + \frac{d-x}{2\pi e} \cdot \delta^{-\frac{2dx}{d-1}} - \frac{d-x}{2\pi e} \cdot \delta^{\frac{2dx}{d-1}} - \frac{x}{2\pi e} \cdot \delta^{\frac{2dx}{d-1} - \frac{2d^2}{d-1}} \end{aligned} \quad (20)$$

Let $\delta^{\frac{2d}{d-1}} = a$ ($a > 1$), then

$$\begin{aligned} \text{Dist}(\Psi_1) - \text{Dist}(\Psi_2) &= \frac{x}{2\pi e} \cdot a^{d-x} + \frac{d-x}{2\pi e} \cdot a^{-x} - \frac{d-x}{2\pi e} \cdot a^x - \frac{x}{2\pi e} \cdot a^{x-d} \\ &= \frac{a^{-x}}{2\pi e} \cdot (x \cdot a^d + d - x - d \cdot a^{2x} + x \cdot a^{2x} - x \cdot a^{-d} \cdot a^{2x}) \\ &= \frac{a^{-x}}{2\pi e} \cdot [x \cdot (1 - a^{-d}) - d] \cdot (a^{2x} + a^d) + d \cdot a^d + d]. \end{aligned} \quad (21)$$

Let $g(x) = [x \cdot (1 - a^{-d}) - d] \cdot (a^{2x} + a^d) + d \cdot a^d + d$, then

$$g'(x) = 2 \ln a \cdot [x \cdot (1 - a^{-d}) - d] \cdot a^{2x} + (1 - a^{-d})(a^{2x} + a^d).$$

Let

$$\begin{aligned} g''(x) &= (2 \ln a)^2 \cdot [x \cdot (1 - a^{-d}) - d] \cdot a^{2x} + 4 \ln a \cdot (1 - a^{-d}) \cdot a^{2x} \\ &= 4 \ln a \cdot a^{2x} \cdot [\ln a \cdot x \cdot (1 - a^{-d}) - d \cdot \ln a + 1 - a^{-d}]. \end{aligned} \quad (22)$$

Since $x \leq y$ and $x + y = d$, it infers that $x \leq d/2$. Since $g''(x)$ is a monotonic incrementing function, $g''(x) \leq g''(d/2)$. $g''(d/2) = 4 \ln a \cdot a^d \cdot [1 - a^{-d} - d/2 \cdot a^{-d} \cdot \ln a - d/2 \cdot \ln a] < 0$, if $\ln a > 0$, $a^d > 0$ and $1 - a^{-d} - d/2 \cdot a^{-d} \cdot \ln a - d/2 \cdot \ln a < 0$. The first two conditions are satisfied obviously because $a > 1$. Now we prove that $1 - a^{-d} - d/2 \cdot a^{-d} \cdot \ln a - d/2 \cdot \ln a < 0$.

Let $h_1(c) = \frac{c^2-1}{c^2+1}$, $h_2(c) = \ln c$, the derivative of $h_1(c)$ and $h_2(c)$ is $h'_1(c) = \frac{4c}{c^4+2c^2+1}$, $h'_2(c) = \frac{1}{c}$. It is obvious that $h_1(1) = h_2(1) = 0$. For $c \geq 1$, $h'_1(c) \leq h'_2(c) \Leftrightarrow \frac{4c}{c^4+2c^2+1} \leq \frac{1}{c} \Leftrightarrow 4c^2 \leq c^4 + 2c^2 + 1 \Leftrightarrow (c^2 - 1)^2 \geq 0$. Thus, $h_1(c) \leq h_2(c)$, $\forall c \geq 1$. The equal sign can be taken if $c = 1$. Let $c = a^{d/2} > 1$, we can obtain that $\frac{a^d-1}{a^d+1} < \frac{d}{2} \ln a \Leftrightarrow a^d - 1 < \frac{d}{2} \cdot a^d \cdot \ln a + \frac{d}{2} \cdot \ln a \Leftrightarrow 1 - a^{-d} - \frac{d}{2} \cdot a^{-d} \cdot \ln a - \frac{d}{2} \cdot \ln a < 0$.

It concludes that $g''(x) < 0, \forall x \leq \frac{d}{2}$. Thus, $g(x)$ is a concave function, and according to the nature of the concave function, $g(x) \geq \min\{g(0), g(d/2)\} = 0$. So $\text{Dist}(\Psi_1) \geq \text{Dist}(\Psi_2)$. $\square$

Lemma 2. Let $x \in \mathbb{Z}^*$ and d be the dimension of the lattice. Then we have

1. (1) $\llbracket x, y \rrbracket <_\gamma \llbracket x-1, y+1 \rrbracket, \forall x \geq 2$; (2) $T_{\text{NC}}(\llbracket x, y \rrbracket) > T_{\text{NC}}(\llbracket x-1, y+1 \rrbracket), \forall d > x > \lceil d/2 \rceil$.
2. (1) $\llbracket x, d-x \rrbracket <_\gamma \llbracket x', d-x' \rrbracket, \forall x > x' \geq 2$; (2) $T_{\text{NC}}(\llbracket x, d-x \rrbracket) > T_{\text{NC}}(\llbracket x', d-x' \rrbracket), \lceil d/2 \rceil \leq x' < x$.
3. In particular, $\forall x > \lceil d/2 \rceil$, we have (1) $\llbracket x, d-x \rrbracket <_\gamma \llbracket \lceil d/2 \rceil, \lfloor d/2 \rfloor \rrbracket$, (2) $T_{\text{NC}}(\llbracket x, d-x \rrbracket) > T_{\text{NC}}(\llbracket \lceil d/2 \rceil, \lfloor d/2 \rfloor \rrbracket)$.
4. For a k -size non-increasing strategy $\Psi = \llbracket \beta_1, \dots, \beta_{z_1}, \dots, \beta_{z_2}, \dots, \beta_k \rrbracket$ and Ψ' , where $\Psi'[z_1] = \beta_{z_1} - 1, \Psi'[z_2] = \beta_{z_2} + 1$ and Ψ' remains non-increasing, $\forall 1 \leq z_1 < z_2 \leq k$. Then we have $\Psi <_\gamma \Psi'$ and $T_{\text{NC}}(\Psi) > T_{\text{NC}}(\Psi')$.

Proof. **(Proof of inequality (1) in item 1).** Let $x, y \in \mathbb{Z}^*$ be integers satisfying $x + y = d$. From Theorem 2, we have

$$\begin{aligned} \text{Dist}^2(\llbracket x, y \rrbracket) &= \text{GH}^2(\mathcal{L}_{\pi[x]}) + \text{GH}^2(\mathcal{L}_{\pi[y]}) \\ &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-1}{d-1})}, \end{aligned} \quad (23)$$

$$\begin{aligned} \text{Dist}^2(\llbracket x-1, y+1 \rrbracket) &= \frac{x-1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-2}{d-1})} + \frac{y+1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-2}{d-1})}. \\ \text{Dist}^2(\llbracket x, y \rrbracket) - \text{Dist}^2(\llbracket x-1, y+1 \rrbracket) &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{y}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-1}{d-1})} - \frac{x-1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-2}{d-1})} - \frac{y+1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{2x+y-2}{d-1})} \\ &= \frac{x}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-1}{d-1})} + \frac{d-x}{2\pi e} \cdot \delta^{-\frac{2dx}{d-1}} - \frac{x-1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-2}{d-1})} - \frac{d-x+1}{2\pi e} \cdot \delta^{-\frac{2d(x-1)}{d-1}} \\ &= \frac{x}{2\pi e} \cdot \delta^{\frac{2d^2}{d-1} - \frac{2dx}{d-1}} + \frac{d-x}{2\pi e} \cdot \delta^{-\frac{2dx}{d-1}} - \frac{x-1}{2\pi e} \cdot \delta^{2d \cdot (1 - \frac{x-2}{d-1})} - \frac{d-x+1}{2\pi e} \cdot \delta^{-\frac{2d(x-1)}{d-1}} \end{aligned} \quad (24)$$

Let $\delta^{\frac{2d}{d-1}} = a$ ($a > 1$), then

$$\begin{aligned} \text{Dist}^2(\llbracket x, y \rrbracket) - \text{Dist}^2(\llbracket x-1, y+1 \rrbracket) &= \frac{x}{2\pi e} \cdot a^{d-x} + \frac{d-x}{2\pi e} \cdot a^{-x} - \frac{x-1}{2\pi e} \cdot a^{d-x+1} - \frac{d-x+1}{2\pi e} \cdot a^{-x+1} \\ &= \frac{a^{-x}}{2\pi e} \cdot [x \cdot a^d + d-x - (x-1) \cdot a^{d+1} - (d-x+1) \cdot a] \\ &= \frac{a^{-x}}{2\pi e} \cdot [x \cdot (a^d - 1)(a-1) + d + a^{d+1} - (d+1) \cdot a]. \end{aligned} \quad (25)$$

Let $a = 1 + \epsilon$ for $0 < \epsilon < 1$, from taylor expansion, we have $d + a^{d+1} - (d+1) \cdot a = d + \left[1 + (d+1) \cdot \epsilon + \frac{(d+1)d}{\epsilon^2} + o(\epsilon^2)\right] - (d+1)(1+\epsilon) = \frac{(d+1)d}{\epsilon^2} + o(\epsilon^2) > 0$. Since $a^{-x} > 0$, $(a^d - 1)(a-1) > 0$ and $x \geq 2$, we have $\text{Dist}^2(\llbracket x, y \rrbracket) > \text{Dist}^2(\llbracket x-1, y+1 \rrbracket)$.

(Proof of inequality (2) in item 1). Since $x > y$, we have

$$\begin{aligned} T_{\text{NC}}(\llbracket x, y \rrbracket) - T_{\text{NC}}(\llbracket x-1, y+1 \rrbracket) &= 2^{cx} + 2^{cy} - 2^{c(x-1)} - 2^{c(y+1)} \\ &= 2^{c(x-1)} \cdot (2^c - 1) + 2^{cy} \cdot (1 - 2^c) \\ &> 2^{cy} \cdot (2^c - 1) + 2^{cy} \cdot (1 - 2^c) > 0. \end{aligned} \quad (26)$$

(Proof of item 2 and item 3). This follows by repeated application of item 1 through the chain:

$$\llbracket x, d-x \rrbracket <_\gamma \llbracket x-1, d-x+1 \rrbracket <_\gamma \dots <_\gamma \llbracket x', d-x' \rrbracket <_\gamma \dots <_\gamma \llbracket \lceil \frac{d}{2} \rceil, \lfloor \frac{d}{2} \rfloor \rrbracket,$$

$$T_{\text{NC}}(\llbracket x, d-x \rrbracket) > T_{\text{NC}}(\llbracket x-1, d-x+1 \rrbracket) > \cdots > T_{\text{NC}}(\llbracket x', d-x' \rrbracket) > \cdots > T_{\text{NC}}(\llbracket \lceil \frac{d}{2} \rceil, \lfloor \frac{d}{2} \rfloor \rrbracket),$$

where each step preserves the inequalities by item 1.

(Proof of item 4). The proof of the time cost inequality is the same as Inequality (26), so we omit it. For the distance bound comparison, we construct a chain of γ -relations via item 1 (1):

$$\begin{aligned} \Psi &= \llbracket \beta_1, \dots, \beta_{z_1}, \beta_{z_1+1}, \dots, \beta_{z_2}, \dots, \beta_k \rrbracket \\ &<_{\gamma} \llbracket \beta_1, \dots, \beta_{z_1} - 1, \beta_{z_1+1} + 1, \dots, \beta_{z_2}, \dots, \beta_k \rrbracket \\ &<_{\gamma} \llbracket \beta_1, \dots, \beta_{z_1} - 1, \beta_{z_1+1}, \beta_{z_1+2} + 1, \dots, \beta_{z_2}, \dots, \beta_k \rrbracket \\ &<_{\gamma} \cdots \\ &<_{\gamma} \llbracket \beta_1, \dots, \beta_{z_1} - 1, \beta_{z_1+1}, \dots, \beta_{z_2-1}, \beta_{z_2} + 1, \dots, \beta_k \rrbracket = \Psi'. \square \end{aligned} \quad (27)$$

Theorem 3. Let $\beta_i \in \mathbb{Z}^*$ for each $i \in \{1, 2, \dots, k\}$ such that

$$\sum_{i=1}^k \beta_i = d \quad \text{and} \quad \beta_i \geq \beta_{i+1} \text{ for all } 1 \leq i \leq k-1.$$

Define the set of blocksize strategies as

$$\text{BS} := \{ \llbracket \beta_{\sigma_1}, \beta_{\sigma_2}, \dots, \beta_{\sigma_k} \rrbracket : (\sigma_1, \sigma_2, \dots, \sigma_k) \text{ is a permutation of } (1, 2, \dots, k) \}.$$

Let $\Psi_0 = \llbracket \beta_1, \beta_2, \dots, \beta_k \rrbracket \in \text{BS}$ denote the ordered blocksize security Estimation based on batch-CVPe strategy. Then, for all $\Psi \in \text{BS}$, the following inequality holds:

$$\text{Dist}(\Psi_0) \leq \text{Dist}(\Psi), \quad (28)$$

with equality if and only if $\Psi = \Psi_0$.

Proof. For the case $k = 2$, Inequality (28) follows directly from Lemma 1.

For the case $k \geq 3$. Let $\Psi_0 = \llbracket \beta_1, \beta_2, \dots, \beta_k \rrbracket \in \text{BS}$, $d_i = \sum_{j=1}^i \beta_j$ for $1 \leq i \leq k$, with $d_0 = 0$. By Theorem 2, we have $\text{Dist}(\Psi) = \sum_{i=1}^k \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}:d_i]})$.

For $\Psi = \llbracket \beta_{\sigma_1}, \beta_{\sigma_2}, \dots, \beta_{\sigma_k} \rrbracket \in \text{BS}$, where only $(\sigma_z, \sigma_{z+1}) = (z+1, z)$ and the other σ_i 's remain the same as i . Then, from Lemma 1, we can get that

$$\begin{aligned} \text{Dist}(\Psi) &= \text{GH}^2(\mathbf{B}_{\pi[d_{z-1}:d_{z-1}+\beta_{z+1}]}) + \text{GH}^2(\mathbf{B}_{\pi[d_{z-1}+\beta_{z+1}:d_{z-1}+\beta_z+\beta_{z+1}]}) \\ &\quad + \sum_{i=1}^{z-1} \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}:d_i]}) + \sum_{i=z+2}^k \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}:d_i]}) \\ &\geq \text{GH}^2(\mathbf{B}_{\pi[d_{z-1}:d_{z-1}+\beta_z]}) + \text{GH}^2(\mathbf{B}_{\pi[d_{z-1}+\beta_z:d_{z-1}+\beta_z+\beta_{z+1}]}) \\ &\quad + \sum_{i=1}^{z-1} \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}:d_i]}) + \sum_{i=z+2}^k \text{GH}^2(\mathbf{B}_{\pi[d_{i-1}:d_i]}) = \text{Dist}(\Psi_0). \end{aligned} \quad (29)$$

For $\Psi = \llbracket \beta_{\sigma_1}, \beta_{\sigma_2}, \dots, \beta_{\sigma_k} \rrbracket \in \text{BS}_{k+1}$, where only $(\sigma_{z_1}, \sigma_{z_2}) = (z_2, z_1)$ with $z_1 \neq z_2$ and the other σ_i 's remain the same as i . Assume $z_1 < z_2$. If $z_2 = z_1 + 1$, then the proof process is the same as above. Otherwise, we can obtain the same conclusion by iteratively proving $\text{GH}(\Psi) \geq \text{GH}(\Psi_1) \geq \text{GH}(\Psi_2) \geq \cdots \geq \text{GH}(\Psi_{2(z_2-z_1-1)}) \geq \text{GH}(\Psi_0)$ by setting

$$\begin{cases} (\sigma_{z_1}, \sigma_{z_1+s_1}, \sigma_{z_2}) = (z_1+1, z_2, z_1), & s_1 = 1, \dots, z_2 - z_1 - 1; s_2 = 0 \\ (\sigma_{z_1}, \sigma_{z_2-s_2}, \sigma_{z_2}) = (z_1+1, z_1, z_2), & s_1 = z_2 - z_1 - 1; s_2 = 1, \dots, z_2 - z_1 - 1 \end{cases}$$

and remain the other σ_i 's the same as i for each $\Psi_{s_1+s_2}$. Thus, we have shown that if Ψ can be obtained from Ψ_0 by applying a finite sequence of transpositions, then the inequality $\text{GH}(\Psi_0) \leq \text{GH}(\Psi)$ holds.

For an arbitrary strategy $\Psi = \llbracket \beta_{\sigma_1}, \beta_{\sigma_2}, \dots, \beta_{\sigma_{k+1}} \rrbracket \in \text{BS}$, the transposition decomposition theorem guarantees that any permutation can be expressed as a product of transpositions. Therefore, the desired inequality can be established iteratively. $\square$

Theorem 4. Among all non-increasing k -size `NColattice` strategies for the same lattice with dimension d , the averaged strategy `AvgStrategy`(d, k) achieves the maximal distance bound with minimal time complexity.

Proof. For the case $k = 2$, it follows directly from item 3 in Lemma 2.

For the case $k \geq 3$. An arbitrary k -size non-increasing strategy $\Psi = \llbracket \beta_1, \dots, \beta_k \rrbracket$ can be transformed to `AvgStrategy`(d, k) in finite steps via a sequence:

$$\Psi = \Psi_0 \rightarrow \Psi_1 \rightarrow \dots \rightarrow \Psi_{k-1} = \text{AvgStrategy}(d, k),$$

where $\Psi_i = \llbracket \beta_1, \dots, \beta_{k-i-1} \rrbracket \cup \text{AvgStrategy}(d - \sum_{j \leq k-i-1} \beta_j, i + 1)$. Now we prove that $\Psi_{i-1} <_\gamma \Psi_i$ and $T_{\text{NC}}(\Psi_{i-1}) > T_{\text{NC}}(\Psi_i)$, $\forall 1 \leq i \leq k-1$.

When $i = 1$, $\Psi_{i-1} <_\gamma \Psi_i$ and $T_{\text{NC}}(\Psi_{i-1}) > T_{\text{NC}}(\Psi_i)$ hold directly because of item 3 in Lemma 2. When $i > 1$, one could increase $\Psi_{i-1}[j]$ to $\Psi_i[j]$ by decreasing β_{k-i-1} , for $j > k-i-1$. From item 4 in Lemma 2, we have $\Psi_{i-1} <_\gamma \Psi_i$ and $T_{\text{NC}}(\Psi_{i-1}) > T_{\text{NC}}(\Psi_i)$. $\square$

B More Security Estimations

C Slicer and Colattice Implementation

Since G6K [21] is currently the most efficient sieving-based practical framework in CPU-version³, we implement the iterative slicer and randomized iterative slicer within G6K in C++/Cython/Python. Additionally, we develop the batch-CVP version of the slicer and Colattice in Python, building upon our underlying C++ implementation of the iterative and randomized iterative slicer. The main implementation efforts lie in: (1) integrating sieve-produced vectors into the iterative slicer as reduction vectors, and (2) leveraging the LSH technique to efficiently locate the nearest vector to a target within the sieving vector set L .

G6K treats the `SimHash.compress` function as the LSH function for filtering vectors close to the target vector $\mathbf{t}$. The filtering sensitivity can be controlled by tuning the `XPC_BUCKET_THRESHOLD` parameter. During the sieving process, the system precomputes LSH values for all vectors $\mathbf{v} \in L$ in advance. To facilitate efficient retrieval of vectors sharing the same LSH value as the target vector $\mathbf{t}$, we organize the vector indices into multiple hash buckets indexed by their corresponding LSH values. In the iterative slicer phase, only a single LSH computation is required for the target vector $\mathbf{t}$ - an operation with constant time complexity. During the iterative slicer phase, only the LSH value of the target vector $\mathbf{t}$ needs to be computed - an $\mathcal{O}(1)$ operation. We then use the `is_reducible_maybe` method to check if $\mathbf{t}$ and $\mathbf{v}$ have matching SimHash values (i.e., their LSH similarity exceeds `XPC_BUCKET_THRESHOLD`).

³ <https://github.com/fplll/g6k>

Table 7. Security Estimation for Dilithium: `naive_classical` Cost Model.

Estimator	Parameter	No d4f			Consider d4f		
		Lv2	Lv3	Lv5	Lv2	Lv3	Lv5
$B = \max\{B_{\mathbf{u}}, B_{\mathbf{z}}\}$		350209	724481	769537	350209	724481	769537
Dilithium Paper	β	423	638	909	-	-	-
	bits	123	186	265	-	-	-
NC Strategy	β	433	655	930	442 (403)	669 (617)	950 (882)
	β_1	464	689	966	435	649	911
	$-\log_2(p_{B,\alpha,\beta_1})$	85.8	131.7	188.9	80.8	124.3	183.5
	bits	139.9	205.2	285.9	131.2	194.0	271.6
Optimized Strategy	β	433	654	930	442 (403)	668 (616)	948 (881)
	β_1	463	691	966	434	651	916
	$-\log_2(p_{B,\alpha,\beta_1})$	92.1	133.6	194.0	82.0	126.1	185.5
	bits	139.8	204.9	285.8	131.1	193.7	271.5
$B = B_{\mathbf{z}}$		130994	524092	524168	130994	524092	524168
NC Strategy	β	473	671	957	483 (442)	686 (633)	976 (907)
	β_1	506	707	992	475	666	942
	$-\log_2(p_{B,\alpha,\beta_1})$	96.0	136.1	195.0	90.1	127.8	185.3
	bits	151.7	210.0	293.7	142.9	198.7	279.2
Optimized Strategy	β	474	671	955	483 (442)	686 (633)	974 (905)
	β_1	503	707	997	477	665	947
	$-\log_2(p_{B,\alpha,\beta_1})$	100.1	136.1	191.5	87.7	133.7	182.2
	bits	151.6	209.9	293.5	142.7	198.5	279.0

β : BKZ block size.

β_1 : The first block of Ψ for `RandNColattice`.

($\cdot$): dimension accounting for “dimensions for free” [15] in BKZ preprocess.

The standard iterative slicer cannot be parallelized because each step depends on finding the closest vector to the current target and performing reduction sequentially, creating an inherent dependency between iterations. In contrast, the randomized slicer processes independent random vectors from the coset $\mathbf{t} + \mathcal{L}$ at each step, allowing for parallel execution. We therefore implement a parallel version of the randomized iterative slicer algorithm. Moreover, when solving batch-CVP problems using this approach, parallelization remains feasible. Consequently, we also present parallelized methods for solving both approximate and exact batch-CVP problems using the slicer algorithm.

D Correctness Verification Experiments

D.1 Time Cost Test for Slicer

Table 8 presents the average time cost of the iterative slicer algorithm across 10 randomly generated CVP instances with dimensions $d \in \{60, 65, \dots, 95\}$ using one and 20 threads on the machine with Intel(R) Xeon(R) Gold 5218 CPU @ 2.30GHz. The experiments were conducted using an input set of short vectors of size $\frac{1}{4} \cdot \left(\frac{4}{3}\right)^{d/2}$ as the default setting in G6K and employing the sieve method `bdg12` in G6K. All the lattice bases are preprocessed by BKZ- β from $\beta = 5$ to 55. From Eq. (11), the theoretical time cost for a single iterative slicer tour is given by $T_{\text{iter.slicer}} = \left(\frac{3}{2\sqrt{2}}\right)^d$ with an approximation factor of $\gamma_1 = \sqrt{4/3}$ when $\alpha = \sqrt{4/3}$ and $T_{\text{pop}} = 1$. This formulation aligns with the experimental setup and provides a theoretical benchmark for the observed results. Besides we also give the test result in Table 8 in running a tour of the randomized slicer to find the approximate closest vector $\mathbf{w}$ within the distance $\gamma \cdot \text{GH}(\mathcal{L})$. “s” is the number of repeated samplings for the iterative slicer by entering a random vector on $\mathbf{t} + \mathcal{L}$. γ is the approximate factor after running the randomized iterative slicer tour (resp. iterative slicer tour).

Table 8. Time Cost of (Randomized) Iterative Slicer Tour on solving CVP $_{\gamma}$

dim	1 Thread (sec)							20 Threads (sec)				$\bar{s}$
	γ	T_{sieve}	$T_{\text{iter.slicer}}$	s	γ	T_{sieve}	$T_{\text{randslicer}}$	s	γ	T_{sieve}	$T_{\text{randslicer}}$	
60	1.156	23.88	0.11	23	1.079	25.02	1.73	30	1.085	16.38	0.60	65
65	1.160	47.29	0.16	19	1.088	51.49	1.88	26	1.074	24.31	0.66	75
70	1.171	79.00	0.21	16	1.078	128.43	2.84	26	1.086	40.19	0.74	86
75	1.169	233.73	0.35	37	1.084	337.34	10.16	50	1.084	69.88	1.71	100
80	1.168	920.09	0.82	52	1.085	937.69	26.82	72	1.086	138.29	3.54	116
85	1.152	3046.30	1.60	61	1.082	2671.56	42.65	68	1.083	279.28	5.22	136
90	1.161	8758.78	3.43	49	1.087	8391.56	88.88	70	1.089	791.88	10.08	182

The left plot of Figure 1 shows that one iterative slicer tour can output a vector close to the approximate factor $\gamma = \sqrt{4/3}$. As the dimension increases, the observed values of γ output by one iterative slicer tour progressively approach the theoretical estimate of $\sqrt{4/3} \approx 1.1547$. The maximum number of samples for the randomized iterative slicer is an adjustable value that can be set freely to find a closer vector. Let

the estimated number of the repeated sampling times (denoted as $\bar{s}$) be $\left(\frac{1}{\mathbb{P}_{\alpha^2, \gamma^2}}\right)^d$ defined in Claim 2 in [1]. Table 8 and the right side of Figure 1 show that the actual sample amount is close to the predicted value when $\gamma = 1.1$ and $\alpha = \sqrt{4/3}$. Since in our experiment, the size of the outputted short vector set is a little bit smaller than $(4/3)^{d/2}$, we recomputed $\mathbb{P}_{\alpha^2, \gamma^2}$ using the exact value of α obtained from the actual size of the short vector set to ensure the accuracy of $\mathbb{P}_{\alpha^2, \gamma^2}$. The $\mathbb{P}_{\alpha^2, \gamma^2}$ can be computed through the open-source code³ provided by the authors of [1].

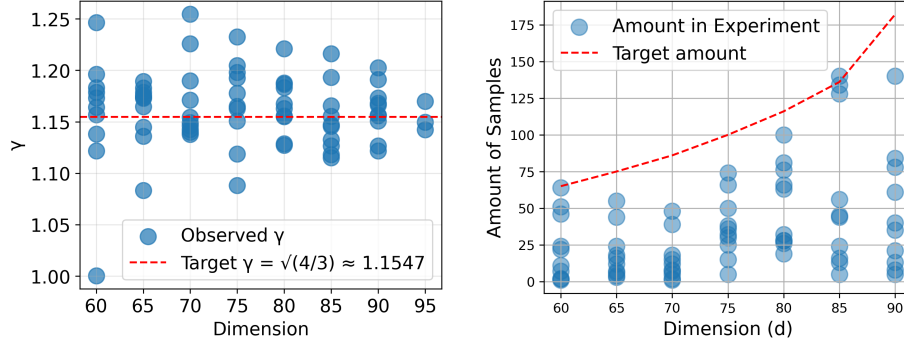


Fig. 1. Approximate factor (resp. Sample amount) output by one (resp. Randomized) iterative slicer call w.r.t sieving dimension using 1 thread.

Since the iterative slicer cannot output the required approximate closest vector while the sieving dimension is not larger than 45, so we use enumeration as the CVP oracle in *NColattice* for small blocksize.

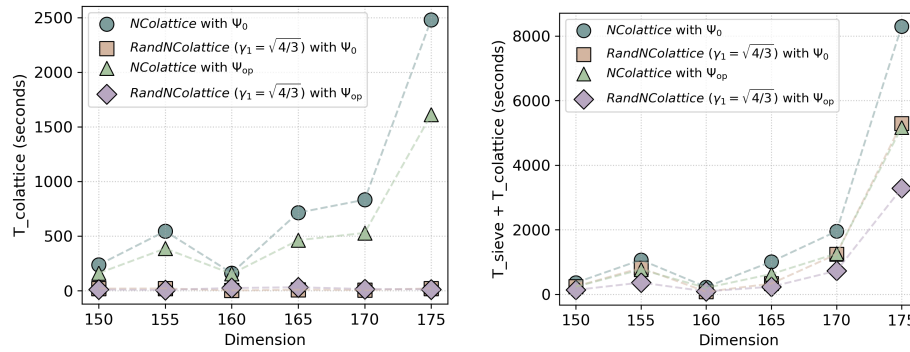


Fig. 2. Time cost for different type of *NColattices*.

³ <https://github.com/WvanWoerden/randomized-slicer>

In Figure 2, we test the average time cost of the variances of **NColattice** to solve CVP_γ with $\gamma = 2.2$ (resp. $\gamma = 2.5$) across 2 (resp. 4) randomly generated CVP instances with dimensions $d \in \{150, 155\}$ (resp. $d \in \{160, 165, 170, 175\}$): (1) the original **NColattice**; (2) **NColattice** with the optimized strategy generated by **NCSSearch**; (3) **RandNColattice** with $\gamma_1 = \sqrt{4/3}$ under the trivial strategy; (4) **RandNColattice** with $\gamma_1 = \sqrt{4/3}$ under the optimized strategy. All the lattice bases are preprocessed by BKZ- β from $\beta = 5$ to 55. The strategies used in **NColattice** are listed in Table 9. “ $\Psi_0 = \llbracket \underbrace{\beta, \dots, \beta}_{\text{repeat } k \text{ times}}, d - k \cdot \beta \rrbracket$ ” means the strategy used in the original **NColattice** by searching a β that can ensure finding the solution. “ Ψ_{op} ” means the strategy generated by **NCSSearch**. “ T_{gen} ” denotes the strategy generation time (in micro seconds).

Table 9. Generate Strategy for **NColattice** with 1 thread

dim	Ψ_0	$T_{\text{gen}}(\text{ms})$	Ψ_{op}	$T_{\text{gen}}(\text{ms})$
150	$\llbracket 68, 68, 14 \rrbracket$	0.6	$\llbracket 69, 41, 40 \rrbracket$	0.6
155	$\llbracket 75, 75, 5 \rrbracket$	0.5	$\llbracket 76, 40, 39 \rrbracket$	0.7
160	$\llbracket 61, 61, 38 \rrbracket$	0.9	$\llbracket 62, 49, 49 \rrbracket$	0.7
165	$\llbracket 70, 70, 25 \rrbracket$	0.8	$\llbracket 71, 47, 47 \rrbracket$	0.7
170	$\llbracket 77, 77, 16 \rrbracket$	0.6	$\llbracket 78, 46, 46 \rrbracket$	0.7
175	$\llbracket 84, 84, 7 \rrbracket$	0.7	$\llbracket 85, 45, 45 \rrbracket$	0.8

Table 10 (resp. Table 11) shows the test result of Randomized Iterative Slicer (resp. **RandNColattice** with $\gamma_1 = \sqrt{4/3}$ and optimized strategy) on batch-CVP $_\gamma$ using 20 threads. The time cost in Table 10 (resp. Table 11) is the **total** time cost for solving the batch-CVP instances with size 100. The number of threads represents the maximum number of threads that can be used by both the sieve method, the randomized iterative slicer and the **RandNColattice**.

Table 10. RandSlicer for batch-CVP $_\gamma$

dim	γ	T_{sieve}	$T_{\text{rand slicer}}$
60	1.1	22.33	47.42
65	1.1	40.20	71.49
70	1.1	63.83	111.51
75	1.1	112.04	210.64
80	1.1	307.28	464.92
85	1.1	785.13	1042.35
90	1.1	2326.73	2223.60

Table 11. RandNColattice($\gamma_1 = \sqrt{4/3}$) for batch-CVP $_\gamma$

dim	γ	T_{sieve}	$T_{\text{RandNColattice}}$
150	2.2	35.30	46.88
155	2.2	78.25	78.76
160	2.5	46.76	78.07
165	2.5	60.60	163.85
170	2.5	121.83	174.62
175	2.5	347.60	280.13